计算机操作系统

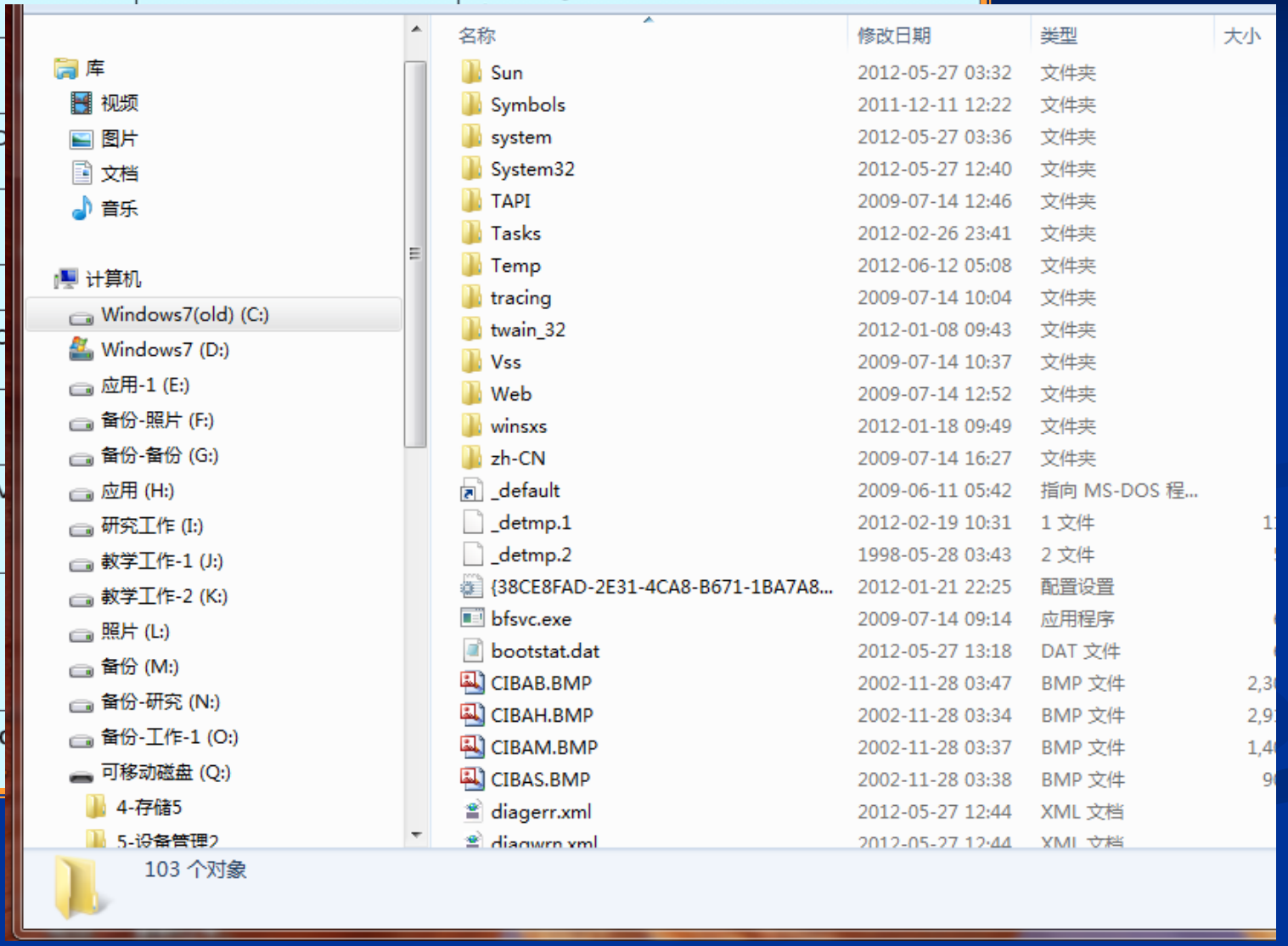
Operating Systems

田卫东

March, 2014

第6章 文件管理

file type	usual extension	function
executable	exe, com, bin	ready-to-run machine-
object		
source c		
batch		
text		
word pro		
library		
print or v		
archive		
multimed		



6.1 文件和文件系统

6.1.1 文件的数据组成

(1) 文件

- 定义

  文件是指由创建者所定义的、具有文件名的若干相关元素的集合。

- 分类

  有结构文件：由若干记录构成的文件；

  无结构(流式)文件：由字节/字符组成的文件；

- 文件属性：文件类型、长度、物理位置、建立时间等

- 示例

  C:\TEST.exe，G:\DOC\个人简历.doc

6.1 文件和文件系统

6.1.1文件的数据组成

(2) 记录

- 定义

  记录是一组相关数据项的集合，用于描述一个对象在某方面的属性。

- 关键字：记录的唯一标志，由单个或组合数据项形成。

- 分类

  定长记录：记录长度固定；

  变长记录：记录长度不定；

- 实例

  张三，19岁，男，95分，78分。

6.1 文件和文件系统

6.1.1文件的数据组成

(3) 数据项

- 定义

  数据项是有结构文件的基本单位，是数据组织中可以命名的最小逻辑单位。也称字段。

  每个数据项包括：名称、数据类型、数据宽度等。

- 分类

  基本数据项：单个数据项；

  组合数据项：若干个数据项集；

- 实例

  姓名，性别。

6.1 文件和文件系统

6.1.2 文件类型

分类方法	分类结果
按用途分类	系统文件，用户文件，库文件
按文件中数据形式分类	源文件，目标文件，可执行文件
按存取控制属性	只执行文件、只读文件、读写文件

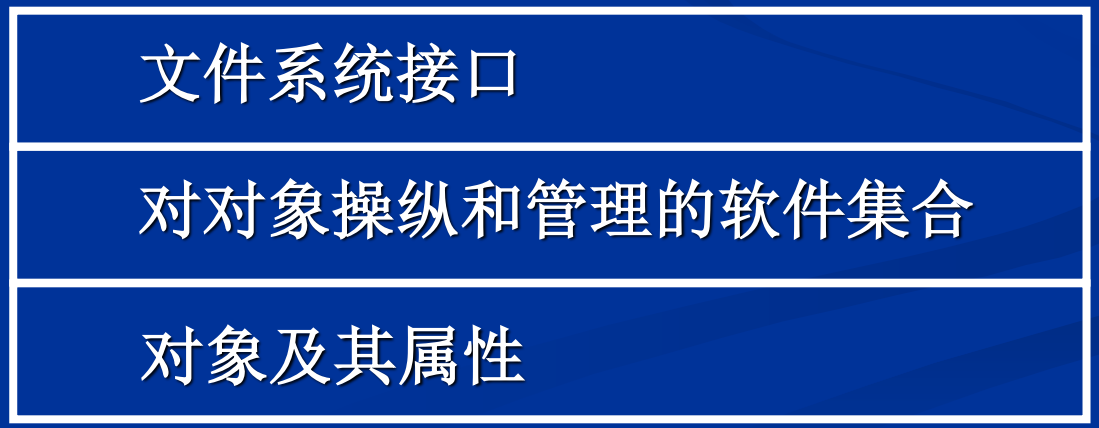
6.1 文件和文件系统

6.1.2 文件系统

- 定义

  文件系统是操作系统中负责管理和存取文件信息的软件机构，它是由管理文件所需的数据结构和相应的管理软件以及访问文件的一组操作组成。

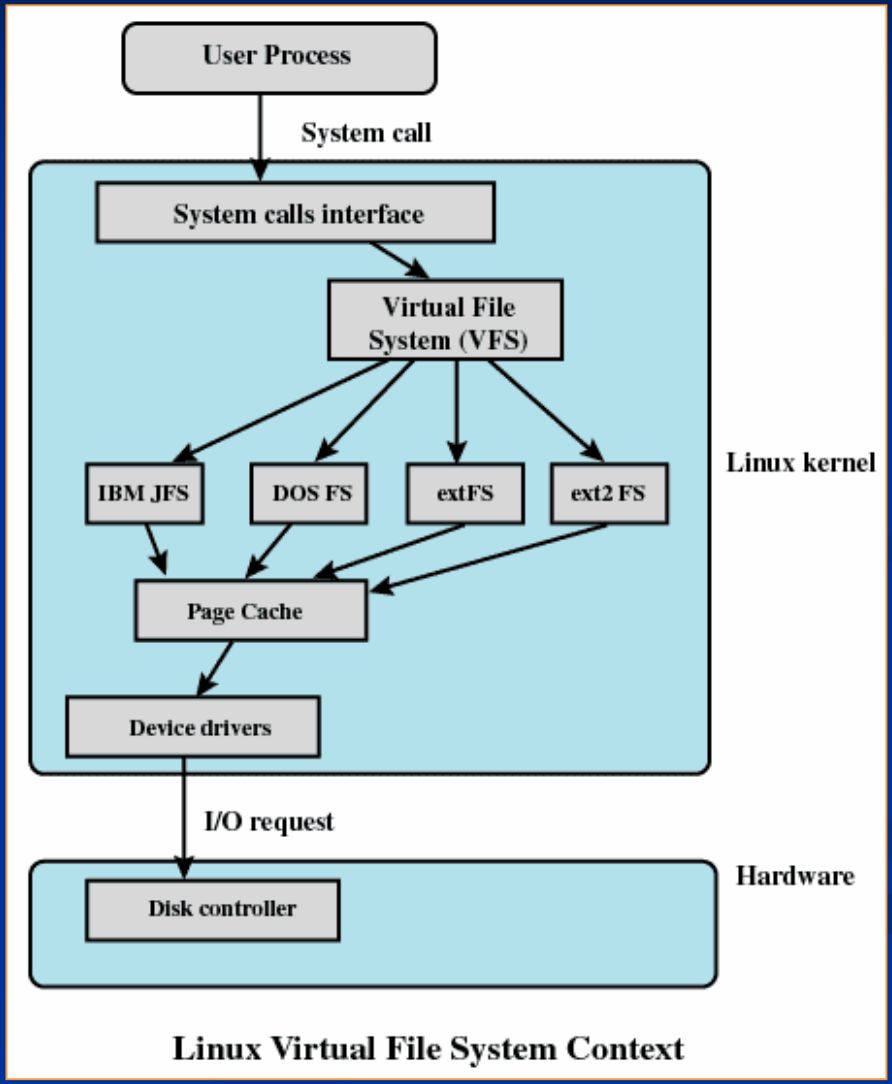
- 示例： NTFS, FAT32, HPFS, ext2, CDFS,ZFS,HFS
- 文件系统模型(三层结构)：



6.1 文件和文件系统

6.1.2 文件系统

- 示例：Linux虚拟文件系统



Linux Virtual File System Context

6.1 文件和文件系统

6.1.4 文件系统操作

- 文件操作三部曲

  打开文件、读写等文件操作、关闭文件。

- 打开文件

  核心工作是，在内存设置对文操作的相关数据结构，并将文件基本信息复制到内存。

- 关闭文件

  核心工作是，删除在内存设置的文件数据结构，并将必要的文件修改信息刷新到磁盘。

6.1 文件和文件系统

6.1.4 文件系统操作

操作类型	功能
文件操作	创建文件
文件操作	删除文件
文件操作	打开文件
文件操作	关闭文件
文件操作	截断文件
文件操作	设置读写位置
目录操作	创建目录
目录操作	删除目录
目录操作	检索目录

6.2 文件的逻辑结构

6.2.1 基本概念

- 如何设计文件系统

  虚拟技术：逻辑文件、物理文件。

- 逻辑文件和物理文件

  逻辑文件：从用户观点出发看到的文件。其结构为文件的逻辑结构，是用户可以直接处理的数据及其结构，独立于文件的物理特性，又称文件组织。

  物理文件：从系统角度出发看到的文件。其结构为文件的存储结构/物理结构，是指文件在外存上的存储组织形式，与存储介质的存储性能、外存分配形式密切相关。

- 逻辑文件到物理文件的映射

  文件目录。

6.2 文件的逻辑结构

6.2.2 顺序文件

- 什么是顺序文件

  顺序文件中的记录，按照一定的顺序存储和组织，如时间顺序、关键字顺序等。通常按顺序方式进行文件的读写。

- 顺序文件的读写（基于读写指针）

  设置读、写指针：指向下一条要读、写记录的起始字节位置。读、写指针在读、写文件的过程中自动变化（由系统负责更新，对程序员透明）。

  打开文件时候：读写指针分别指向文件中首字节位置(为0) 。

  Wptr := 0 ;　Rptr := 0 ;

  后续读写文件时：读写指针(定长记录和变长记录文件）变化方式：

  Wptr := Wptr + L_i ; Rptr := Rptr + L_i ;// Li为记录i长度(变长记录)

  Wptr := Wptr + L ; Rptr := Rptr + L ; // L为记录长度(定长记录)

- 顺序文件：有结构文件

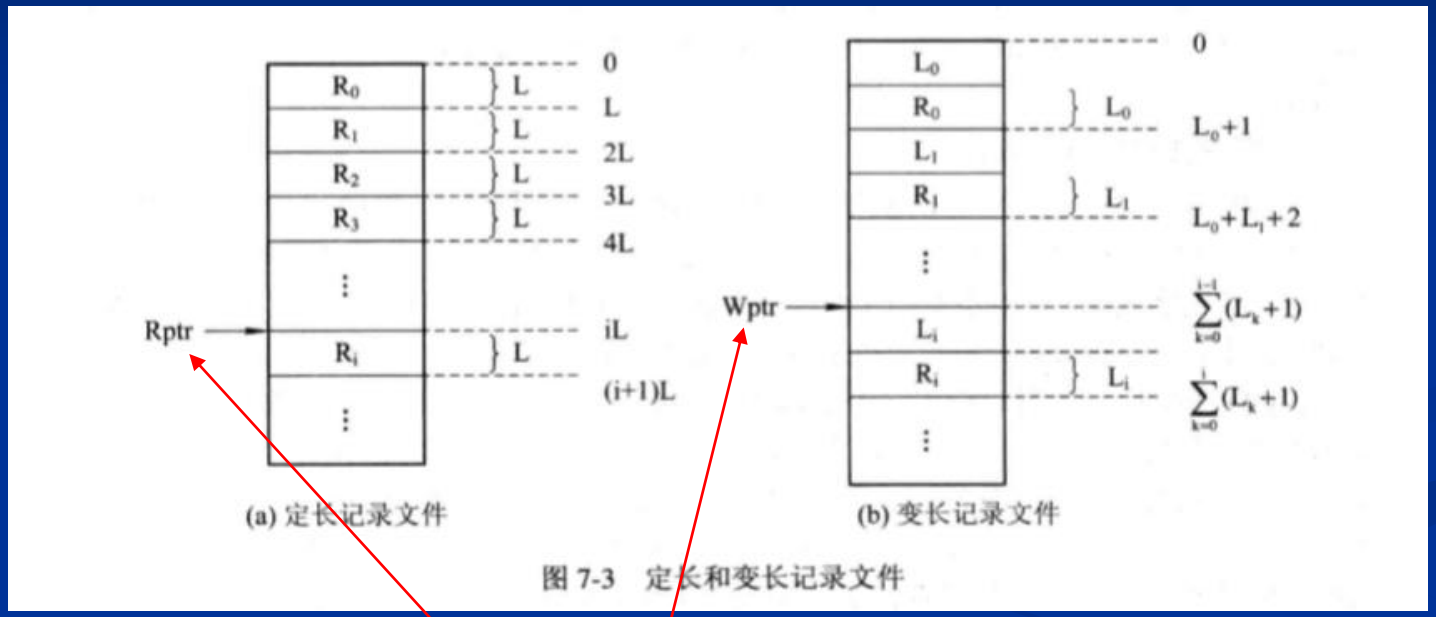


图 7-3　定长和变长记录文件

- 读/写指针：**Rptr, Wptr**

6.2 文件的逻辑结构

6.2.2 顺序文件

- 特点
  - 批量顺序存取，效率非常高。例如，每月工资计算。
  - 查找或修改单条记录困难(记录变长)。例如，查找身份信息，
  - 插入删除记录，困难(记录变长，或须保序) 。

6.2 文件的逻辑结构

6.2.3 索引文件

- 什么是索引文件

定长记录的顺序文件，根据记录号可直接存取，其地址为：

$A_i = i * L$；　　// L为记录长度，i为记录号。

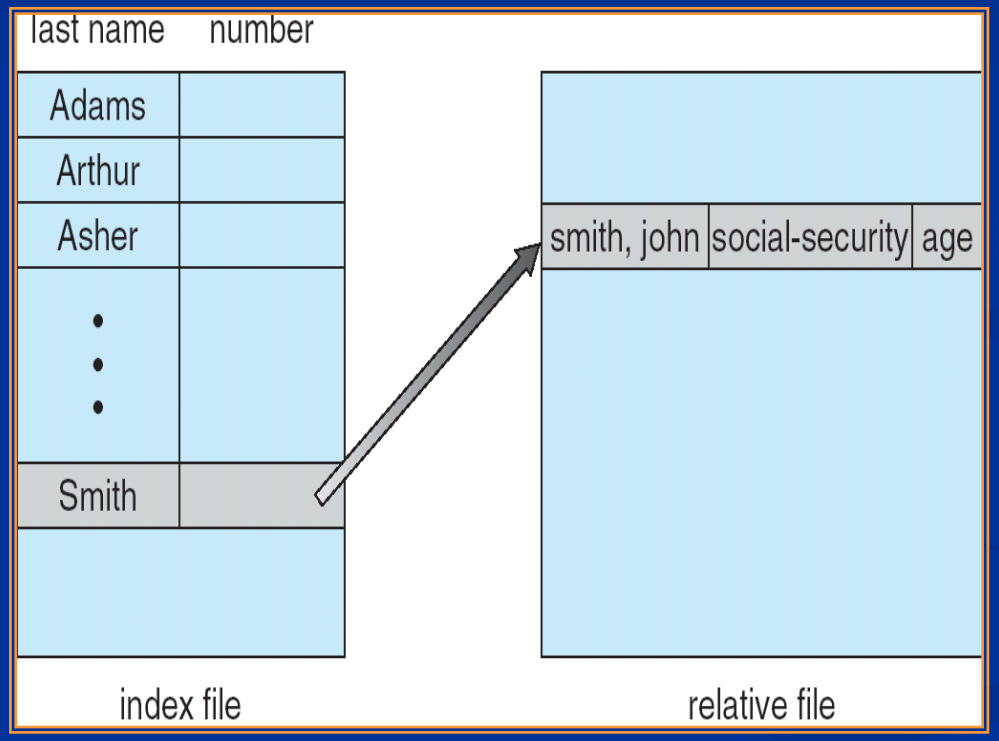
但根据记录的关键字或对于变长记录文件，无法直接存取。

设计思想：在顺序文件基础上，增加索引表，其中存放每条记录的关键字（或记录号）与其文件位置的对应关系。

6.2 文件的逻辑结构

6.2.3 索引文件

- 关键字索引表(也可以是记录号索引表)示例：



- 索引表本身为定长记录结构，支持折半查找。

6.2 文件的逻辑结构

6.2.3 索引文件

- 文件读写

  首先根据索引表，由关键字（记录号）找到记录在文件位置，然后调整读写指针，再进行记录读写。可折半查找。

- 特点
  - 支持记录直接存取；
  - 但索引表须占用额外存储空间，尤其当记录数量很大时。

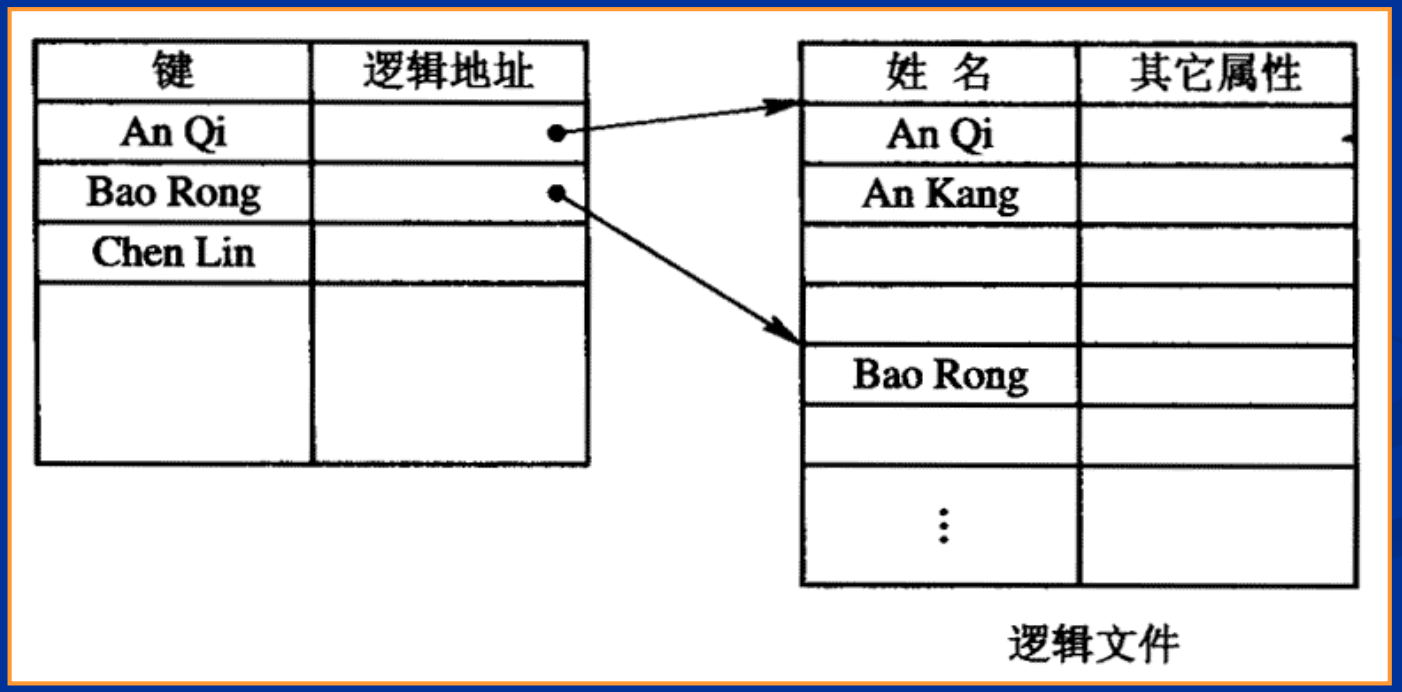
6.2 文件的逻辑结构

6.2.3 索引顺序文件文件

- 什么是索引顺序文件

为降低索引表的存储开销，为一组记录在索引表中建立一个索引项，而非每条记录一条索引项。

分组方式：按关键字分组，或直接按记录号分组。



6.2 文件的逻辑结构

6.2.3 索引顺序文件

- 文件读写

  首先根据索引表，由关键字（记录号）找到记录所在组的文件起始位置，然后调整读写指针，再顺序读写记录，直到找到所要读写的记录。

- 特点
  - 支持记录直接存取；
  - 大大降低索引表的存储空间和I/O、计算开销。

6.2 文件的逻辑结构

6.2.4 直接文件

- 什么是直接文件(HASH文件）

  直接根据记录的关键字（记录号），经简单计算，得到记录在文件的位置。

- 计算记录位置的方式

  A_i = HASH(i) ;　　// 针对记录号

  A_i = HASH(key) ;　　// 针对关键字

- 文件读写

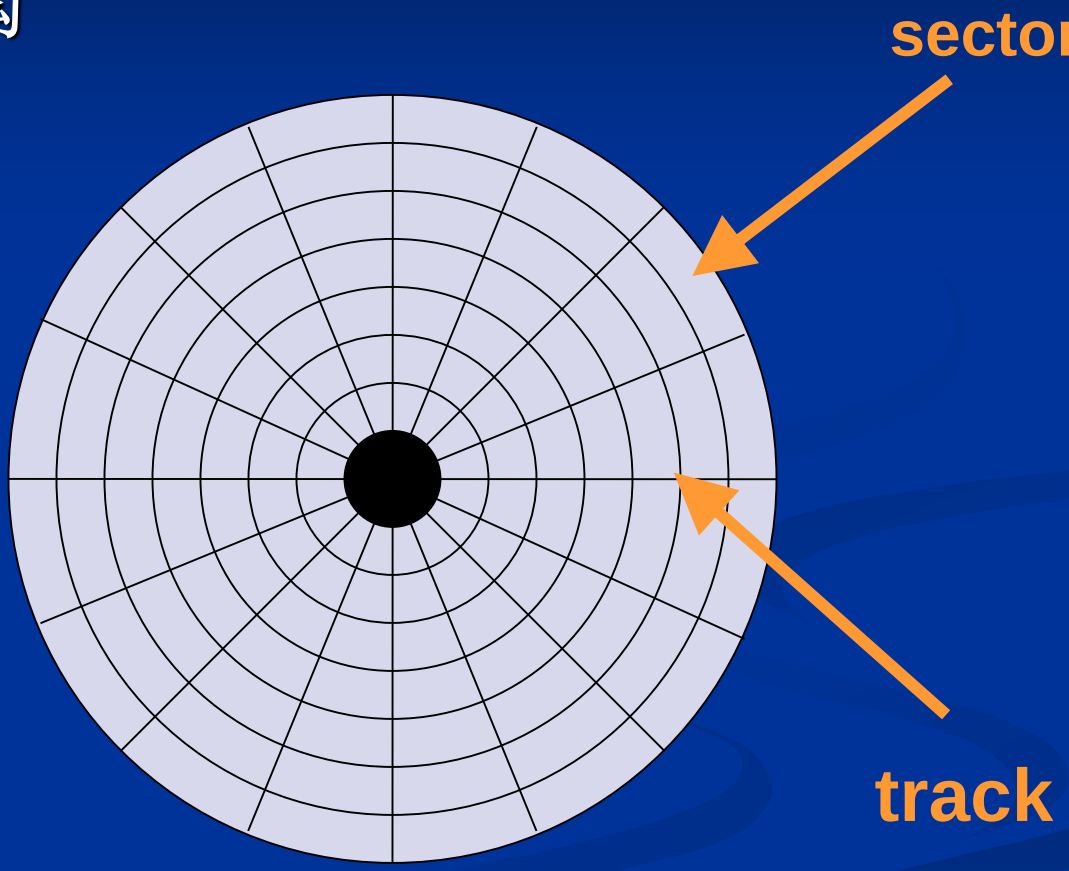
  首先根据HASH函数，由关键字（记录号）计算记录在文件起始位置，然后调整读写指针，再读写。

- 特点
  - 支持记录直接存取；
  - 摆脱索引表，存储空间和I/O、计算开销大幅降低。

6.3 文件的物理结构：外存分配方式

- 磁盘上的文件的物理结构





- 磁盘格式化的工作：(1) 设置磁盘块大小；(2)为磁盘块编号；

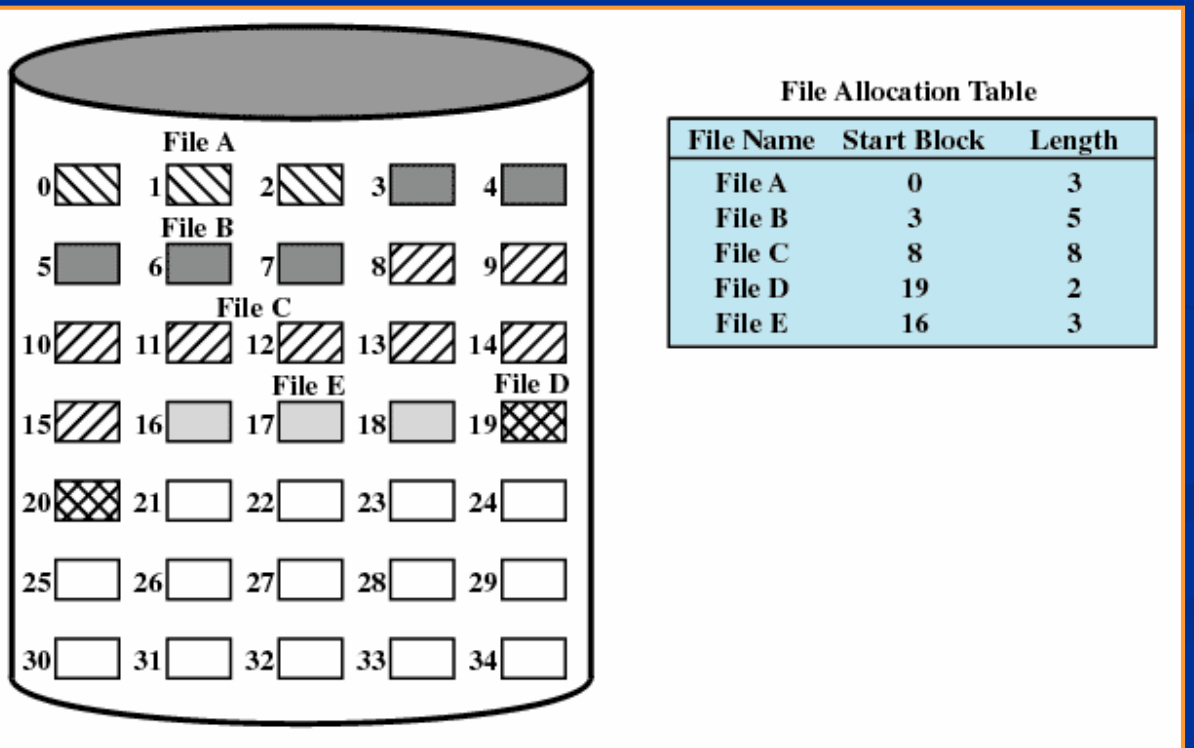
6.3 文件的物理结构：外存分配方式

6.3.1 连续分配

- 什么是连续分配：顺序文件

要求为每个文件分配一组相邻接的磁盘块，且文件的逻辑记录的顺序与所存储磁盘块的块号顺序一致。

所形成的文件结构称顺序文件结构，物理文件称为顺序文件。



File Allocation Table

File Name	Start Block	Length
File A	0	3
File B	3	5
File C	8	8
File D	19	2
File E	16	3

6.3 文件的物理结构：外存分配方式

6.3.1 连续分配

- 特点
  - 顺序访问容易,顺序访问速度快；

    通常，它们位于一条磁道上，在进行读 / 写时，不必移动 磁头，仅当访问到一条磁道的最后一个盘块后，才需要移到下一条磁道，于是又去连续地读 / 写多个盘块。
  - 支持逻辑记录直接存取；
  - 要求连续的存储空间；
  - 必须事先知道文件长度；

6.3 文件的物理结构：外存分配方式

6.3.2 链接分配

- 基本思想

采用离散分配思想，为每个文件分配一组不相邻接的磁盘块，且文件的逻辑记录的顺序与所存储磁盘块的块号顺序可不一致。

核心问题：如何设计数据结构，记录文件分配到的磁盘块信息？

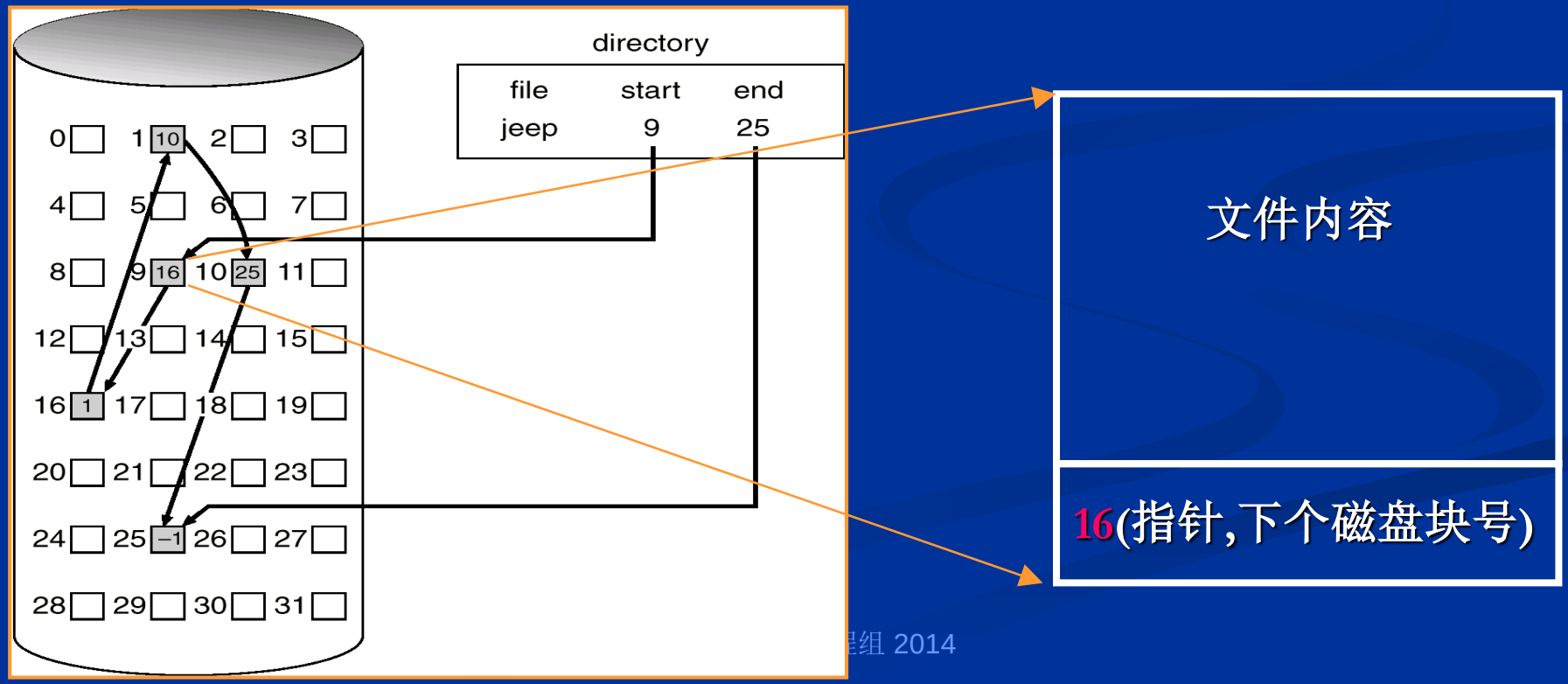
6.3 文件的物理结构：外存分配方式

6.3.2 链接分配

(1) 隐式链接：链接文件

采用单链表结构，各磁盘块存储指向下个磁盘块的块号。

所形成的文件结构称链接文件结构，物理文件称为链接文件。



6.3 文件的物理结构：外存分配方式

6.3.1 链接分配

- 隐式链接特点
  - 不存在外部碎片问题
  - 有利于文件插入和删除
  - 有利于文件动态增长和减小
  - 读写文件时更多的寻道次数和寻道时间，存取速度慢
  - 不适于随机存取
  - 可靠性容错性差，如指针出错

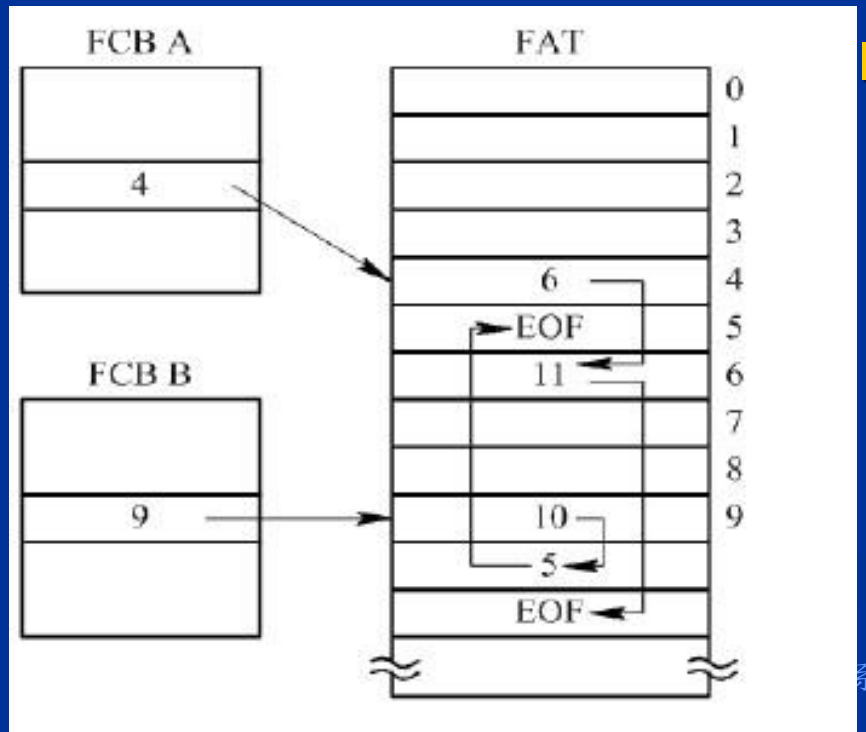
6.3 文件的物理结构：外存分配方式

6.3.2 链接分配

(2) 显式链接

设置文件分配表(File Allocation Table, FAT),集中存储所有的磁盘块号信息。

- FAT表：磁盘块号（簇号）数组。



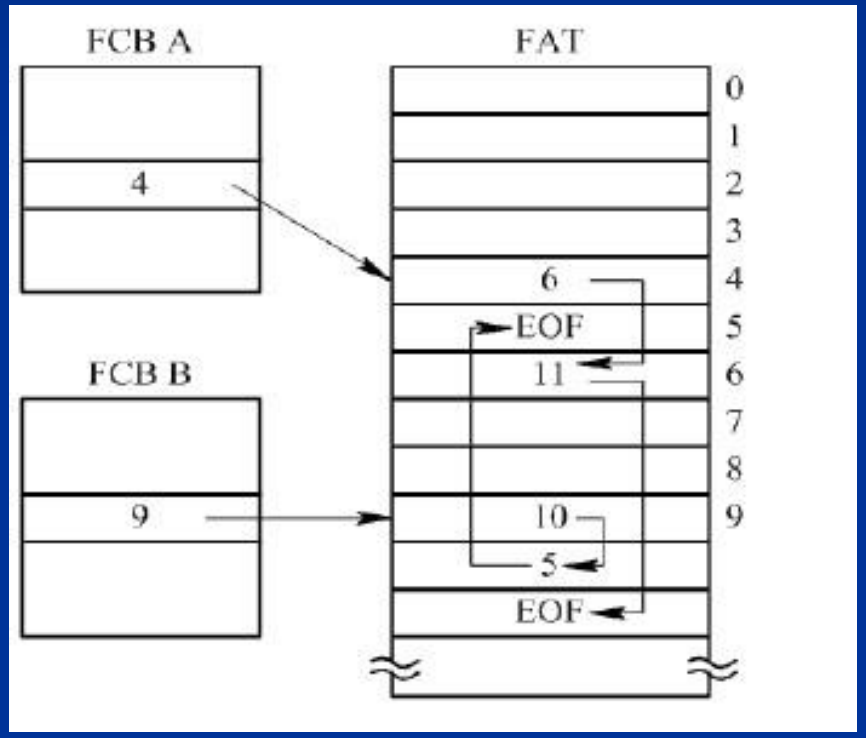
- 示例：
  - 文件A,占据磁盘块为4,6,11。
  - 文件B,占据磁盘块为9,10,5。

核心：单链表的集中存储。

6.3 文件的物理结构：外存分配方式

6.3.2 链接分配

- FAT表：磁盘块号（簇号）数组。



- FAT分类：
  - FAT12

    12位磁盘块号：[0，2^{12}-1]
  - FAT16

    16位磁盘块号：[0，2^{16}-1]
  - FAT32

    32位磁盘块号：[0，2^{32}-1]

6.3 文件的物理结构：外存分配方式

6.3.2 链接分配

- FAT表的磁盘空间管理能力

FAT类型	占用空间	磁盘块(簇)尺寸	管理磁盘空间
FAT12	1.5*4K = 6K	0.5K	2M
		1K	4M
FAT16	2*64K = 128K	1K	64M
		4K	256M
		16K	1G
		32K	2G
		64K	4G
FAT32	4*4G = 16G	1K	4T
	4*4M = 16M	1K	4G
	4*4M = 16M	4K	16G
	4*16M= 64M	4K	64G

6.3 文件的物理结构：外存分配方式

6.3.2 链接分配

- 显式链接特点
  - 不存在外部碎片问题
  - 支持记录的插入和删除
  - 有利于文件动态增长和减小
  - 读写文件时更多的寻道次数和寻道时间，存取速度慢
  - 支持随机存取
  - 安全性容错性比隐式链接好

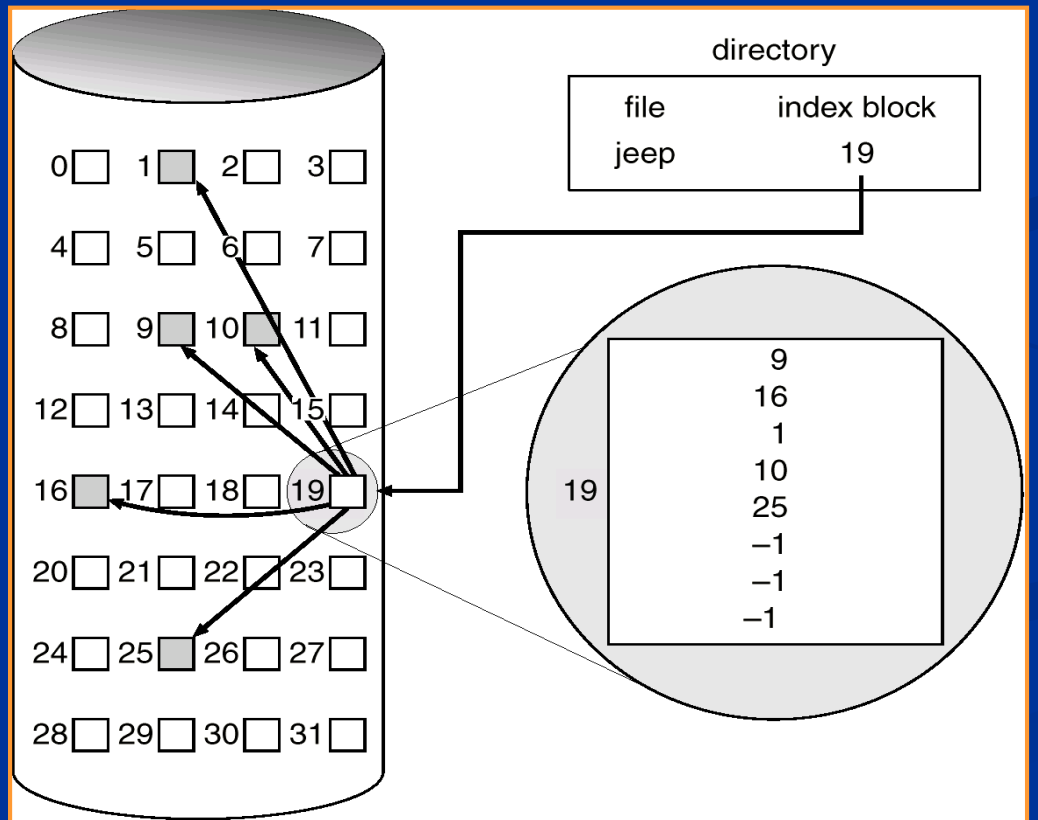
6.3 文件的物理结构：外存分配方式

6.3.3 索引分配

- 什么是索引分配：索引文件

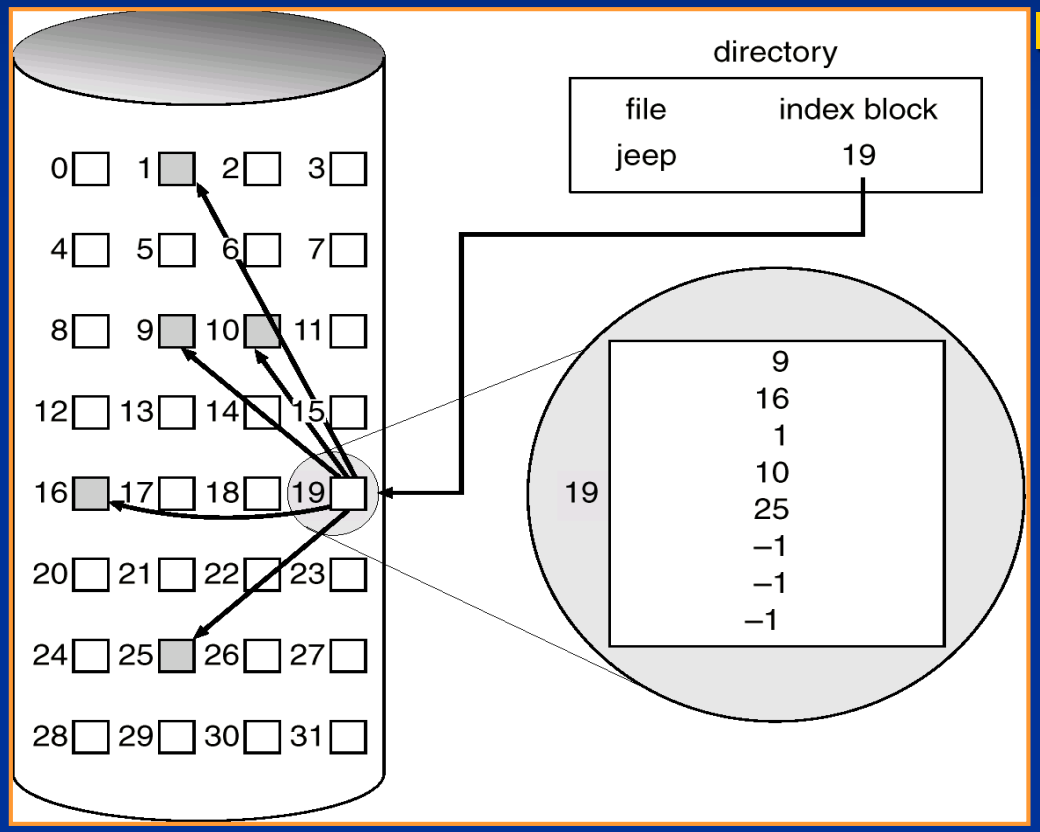
  为每个文件设置索引表，存储其分配到的磁盘块号。

  所形成的文件结构称索引文件结构，物理文件称为索引文件。



6.3 文件的物理结构：外存分配方式

6.3.3 索引分配



- 单级索引：索引表的存储空间

假设磁盘块尺寸为4KB，磁盘块号为32bits。

则每个磁盘块可以存储磁盘块号的数量为：

4K/4 = 1K ；

而单级索引支持的文件尺寸为：

1K * 4K = 4 M。

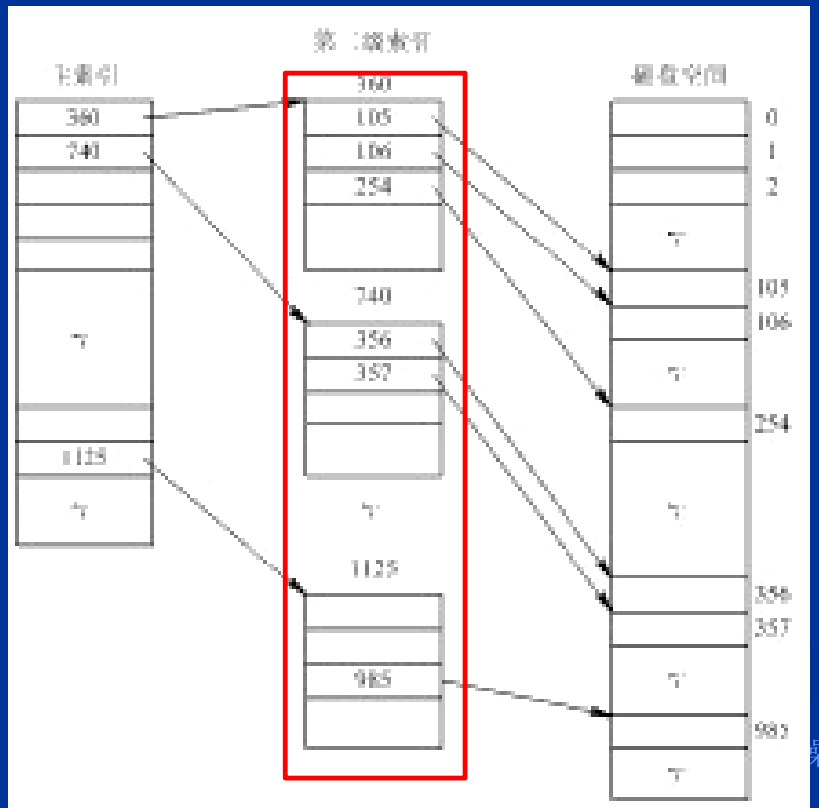
即小于4M文件的索引表只占1磁盘块。

6.3 文件的物理结构：外存分配方式

6.3.3 索引分配

- 二级和多级索引

对于索引表的存储，也采用索引表来记录其所占用的磁盘块号。从而形成二级索引。



- 二级索引：索引表的存储空间

假设磁盘块尺寸为4KB，磁盘块号为32bits。

则每个磁盘块可以存储磁盘块号：

4K/4 = 1K （个）；

而二级索引支持的文件尺寸为：

1K * 1K * 4K = 4G。

索引表占用空间：

4K + 1K*4K

6.3 文件的物理结构：外存分配方式

6.3.3 索引分配

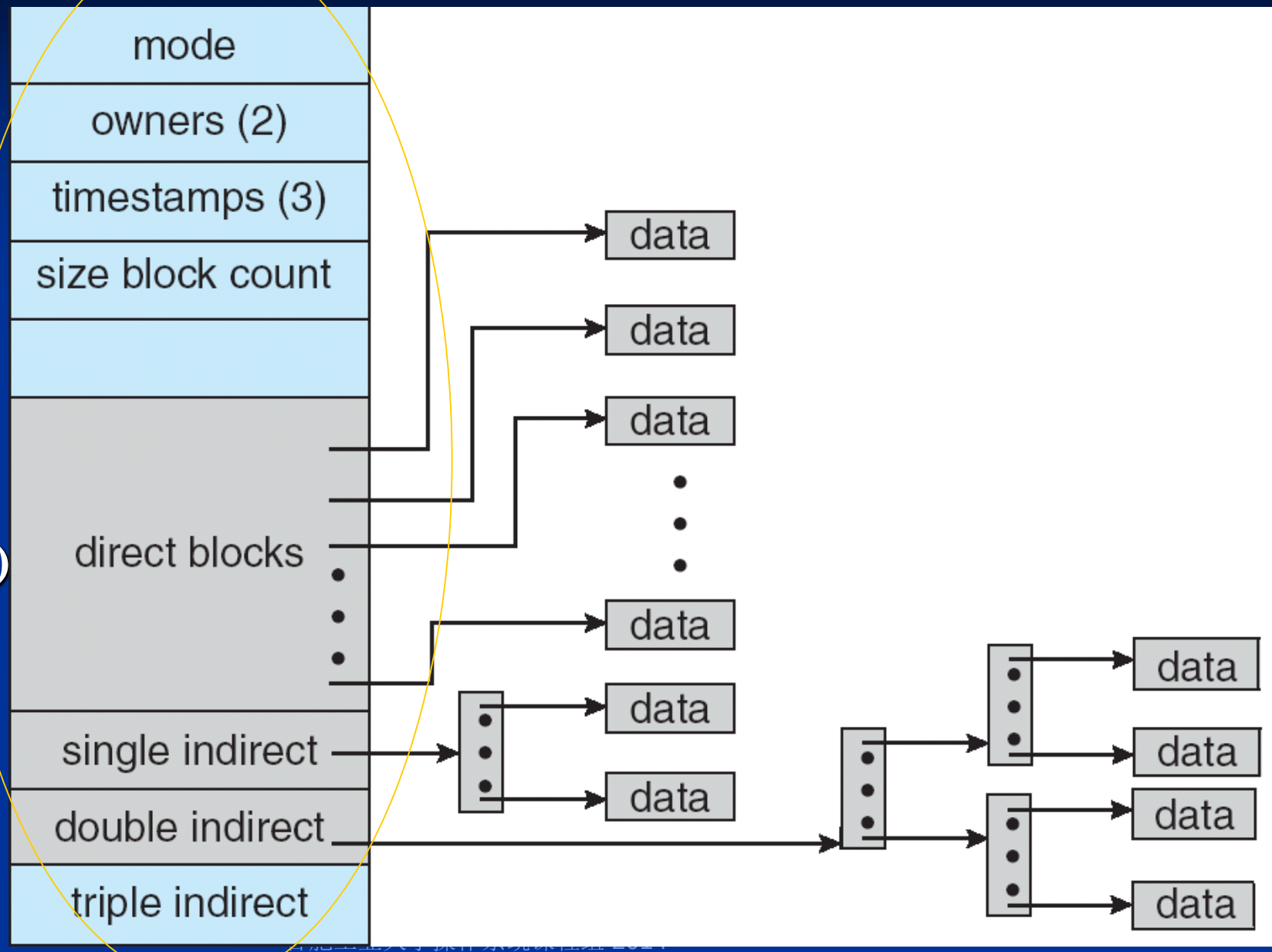
- 混合索引分配方式（UNIX System V）

  混合使用直接磁盘块号、各级索引表，从而可以既支持小文件的存储，也可以支持大文件的存储。

  达到降低索引表存储空间和同时支持大中小文件的目的。

■ 混合索引分配方式（UNIX System V）

索引结点：
(i_node)



i.addr(0)

直接地址(10个)

i.addr(9)

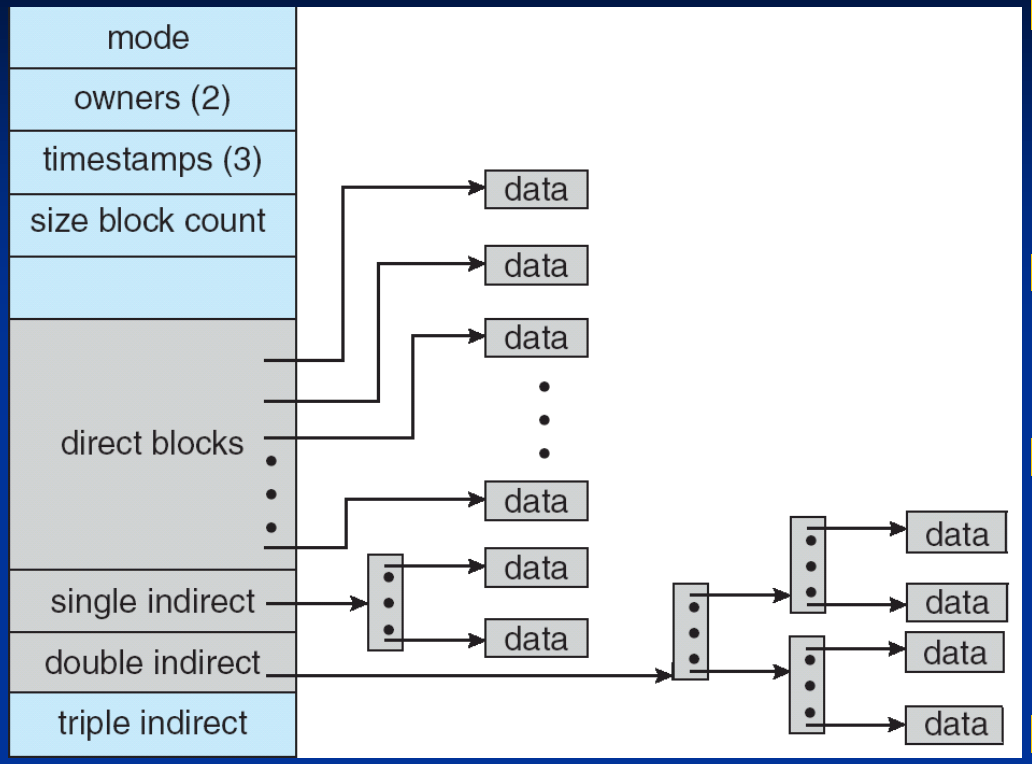
一次间接地址

二次间接地址

三次间接地址

i.addr(12)

混合索引分配方式支持的文件尺寸



- 假设参数

  磁盘块尺寸为4KB，磁盘块号为32bits。
- 直接地址, 支持文件尺寸：

  10*4K = 40K ;
- 一次间接地址, 支持文件尺寸：

  支持文件：1K*4K = 4M ;

  总计：40K + 4M ;
- 二次间接地址, 支持文件尺寸：

  支持文件：1K*1K*4K = 4G ;

  总计：40K + 4M + 4G;
- 三次间接地址, 支持文件尺寸：

  1K*1K*1K*4K = 4T ;

  总计：40K + 4M + 4G + 4T;

6.3 文件的物理结构：外存分配方式

6.3.3 索引分配

- 索引分配的特点
  - 不存在外部碎片问题
  - 读写文件时较多的寻道次数和寻道时间，访问速度慢
  - 支持记录的插入和删除，
  - 有利于文件动态增长和减小
  - 既能顺序存取，又能随机存取
  - 索引表本身带来了系统开销(空间，时间)

6.4 目录管理

- 对文件目录管理的基本要求
  - “按名存取”， 逻辑文件名称
  - 提高文件目录检索速度
  - 文件共享
  - 允许文件重名

6.4 目录管理

6.4.1 文件控制块(FCB, File Control Block)

- 什么是文件控制块

  描述文件信息（文件属性）的数据结构，操作系统据此管理和读写文件。

- 文件控制块包含的信息
  - 基本信息类：

    文件名，文件物理位置，文件逻辑结构，文件物理结构
  - 存取控制信息类：

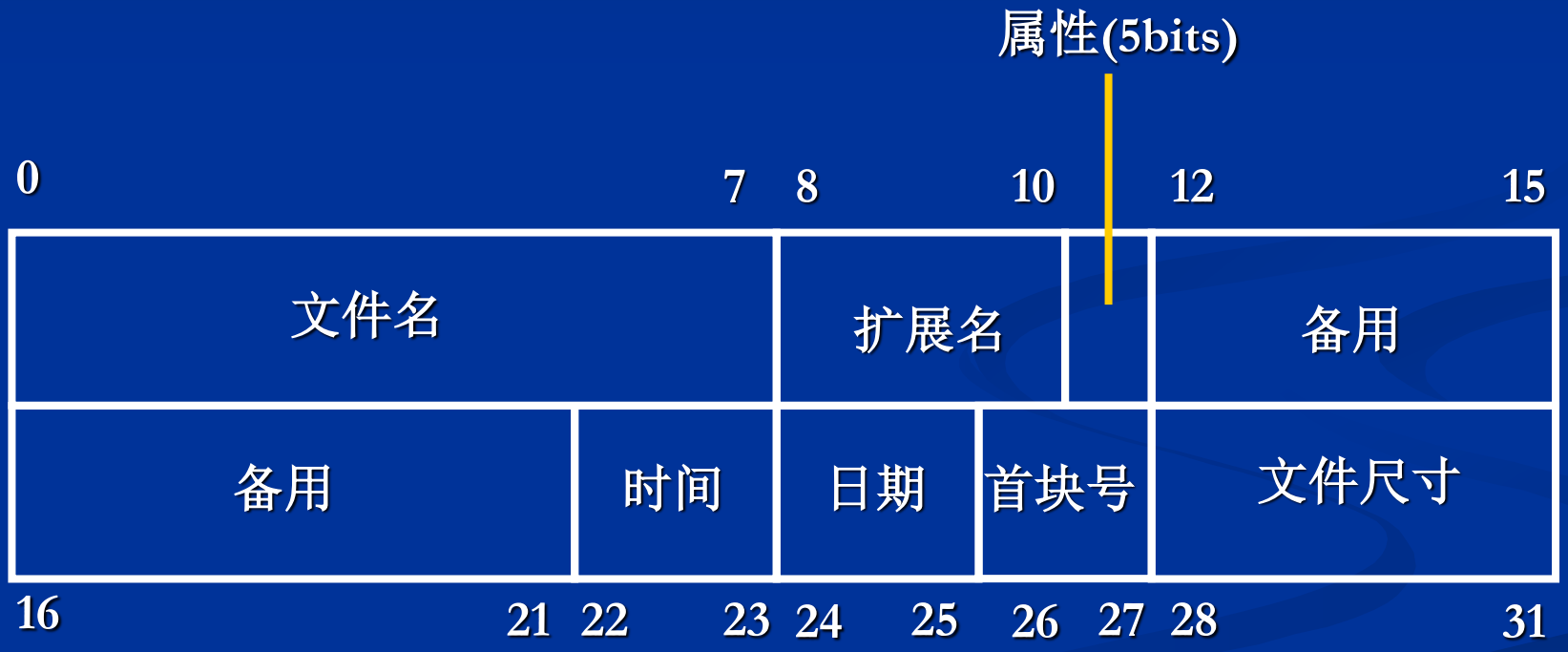
    各类用户（文件主、核准用户等）存取权限
  - 使用信息类：

    文件的建立、修改、访问的日期时间，文件当前使用信息

6.4 目录管理

6.4.1 文件控制块(FCB, File Control Block)

- 典型实例：**MS-DOS的文件控制块（FAT12/16)**



6.4 目录管理

6.4.1 文件控制块(FCB, File Control Block)

- 典型实例：**MS-DOS的文件控制块（FAT16)**

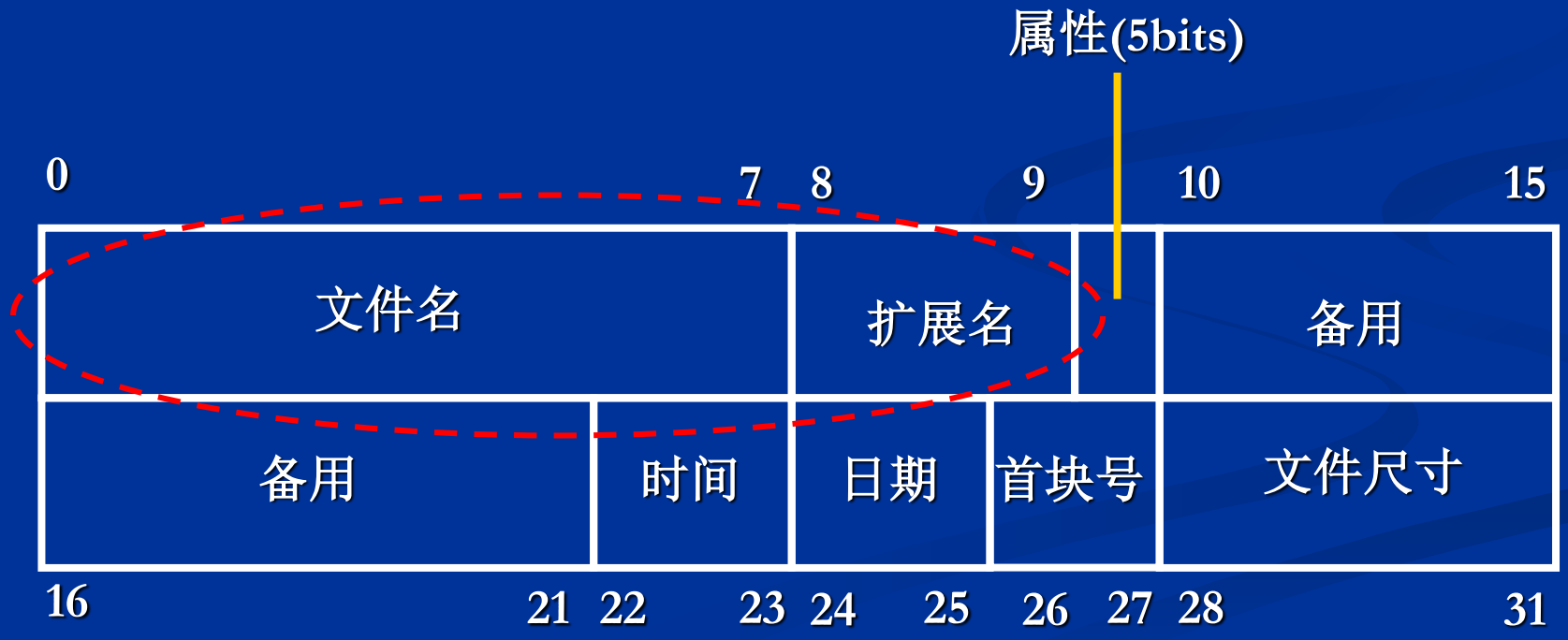
```
struct FCB_FAT16 {
    char filename[8];
    char ext[3];
    char attribute;
    char reserved[10];
    short int time;
    short int date;
    short int firstblock;
    int  size ;
};
```

6.4 目录管理

6.4.1 文件控制块(FCB, File Control Block)

- 文件目录：可看作是文件控制块数组

  检索效率问题：FCB中大量信息在检索中无用，但是占用I/O带宽和存储空间。

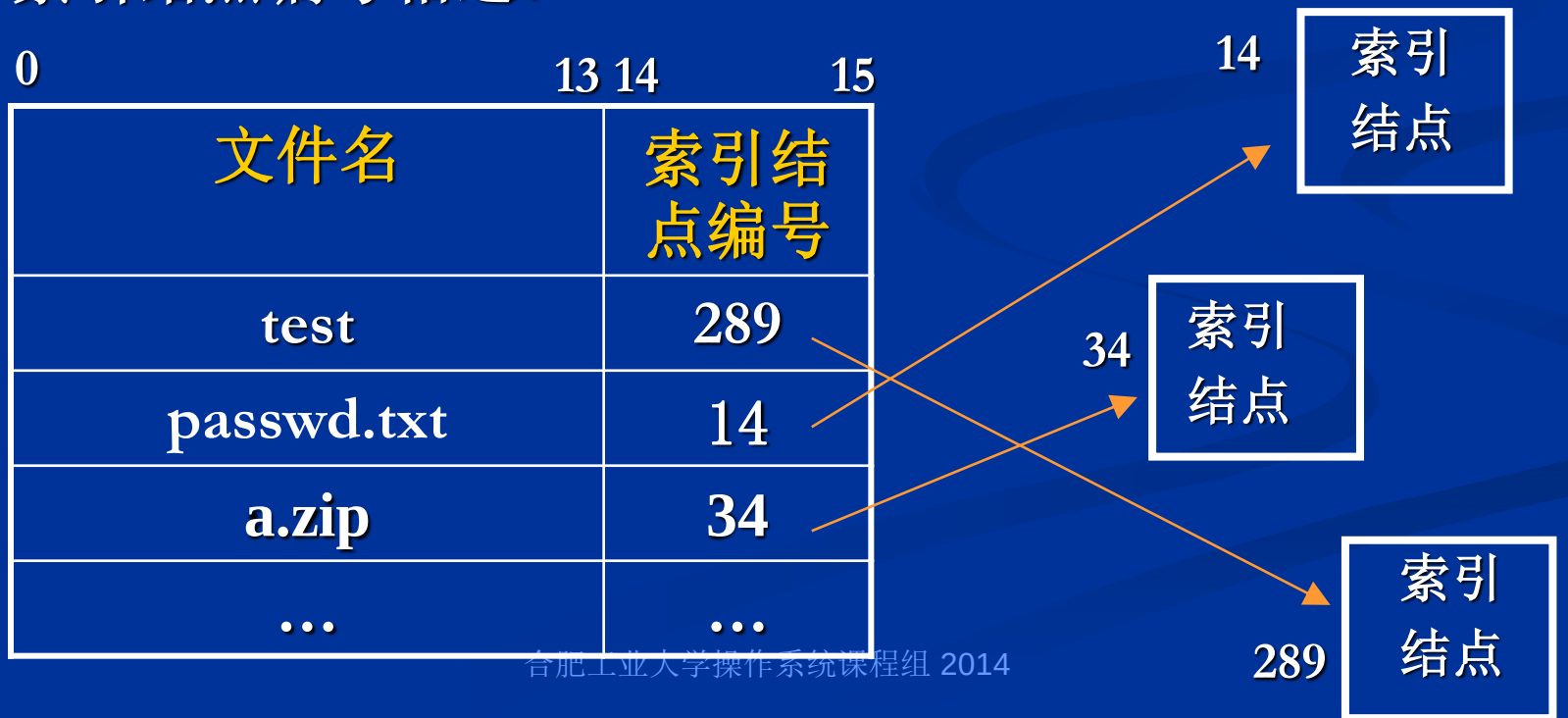


6.4 目录管理

6.4.1 文件控制块(FCB, File Control Block)

- 索引结点（i_node）

  UNIX系统，将FCB中的文件名和文件的其余属性信息分开，前者用于组成文件目录，后者形成索引结点。两者之间通过索引结点编号相连。



6.4 目录管理

6.4.1 文件控制块(FCB, File Control Block)

- 内存索引结点和磁盘索引结点
  - 磁盘索引结点：

    存放在磁盘，存储除文件名外的文件属性信息。
  - 内存索引结点：

    存放在内存，除包含复制的磁盘索引结点信息外，还有：

    (I) 索引结点编号，状态，访问计数，

    (II)文件所属文件系统的逻辑设备号，链接指针。

6.4 目录管理

6.4.2 目录结构

(1) 单级目录结构

整个文件系统中只建立一张目录表（组成一线性表），每文件占用一个目录项。

- 特点
  - 查找速度慢；
  - 不允许重名；
  - 不便于文件共享；

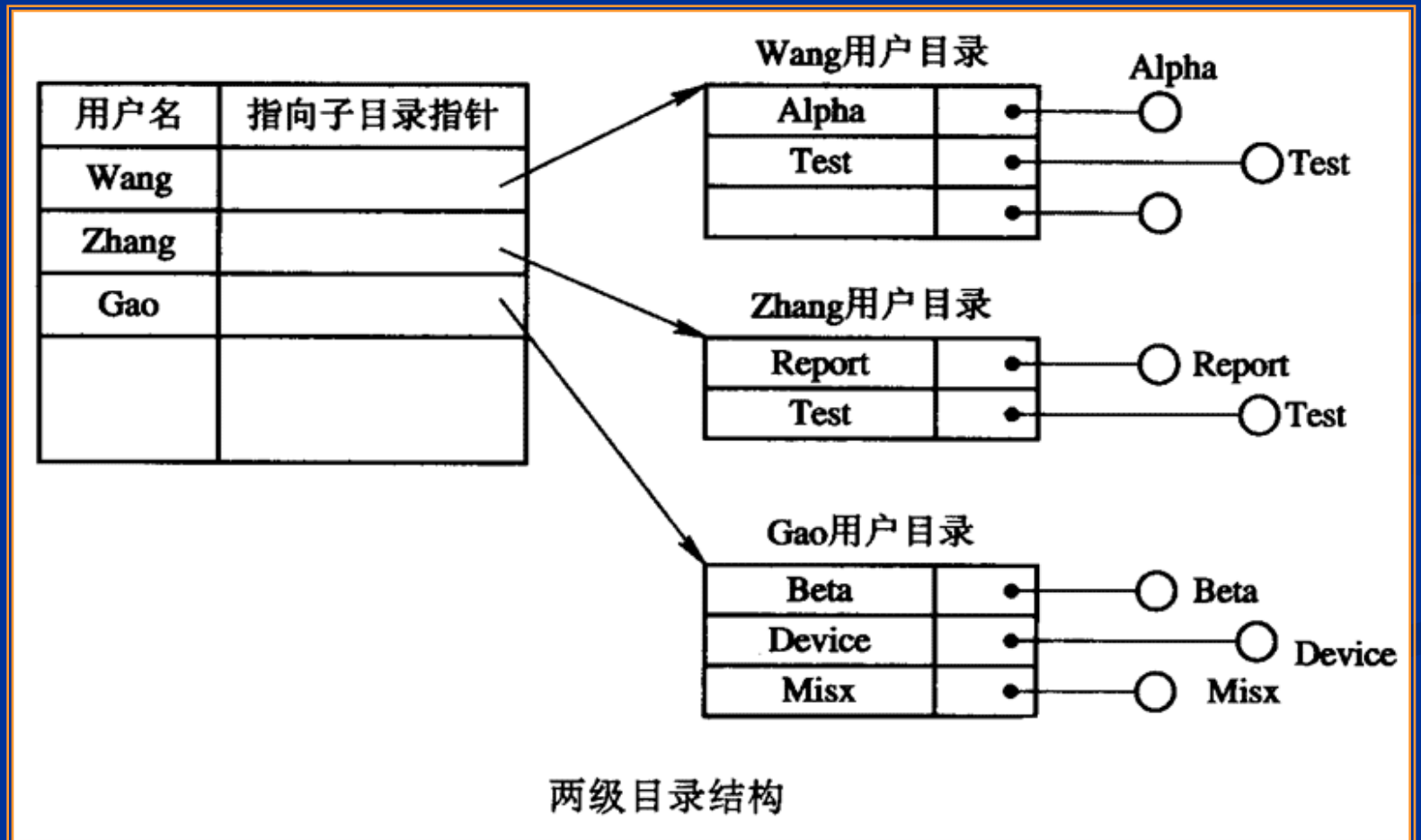
- 单级文件目录：

FCB0
FCB1
FCB2
FCB3
FCB4
…
FCBn

6.4 目录管理

(2) 二级目录结构

在主文件目录基础上，允许每个用户建立一个目录，文件存放在各用户子目录下。



两级目录结构

6.4 目录管理

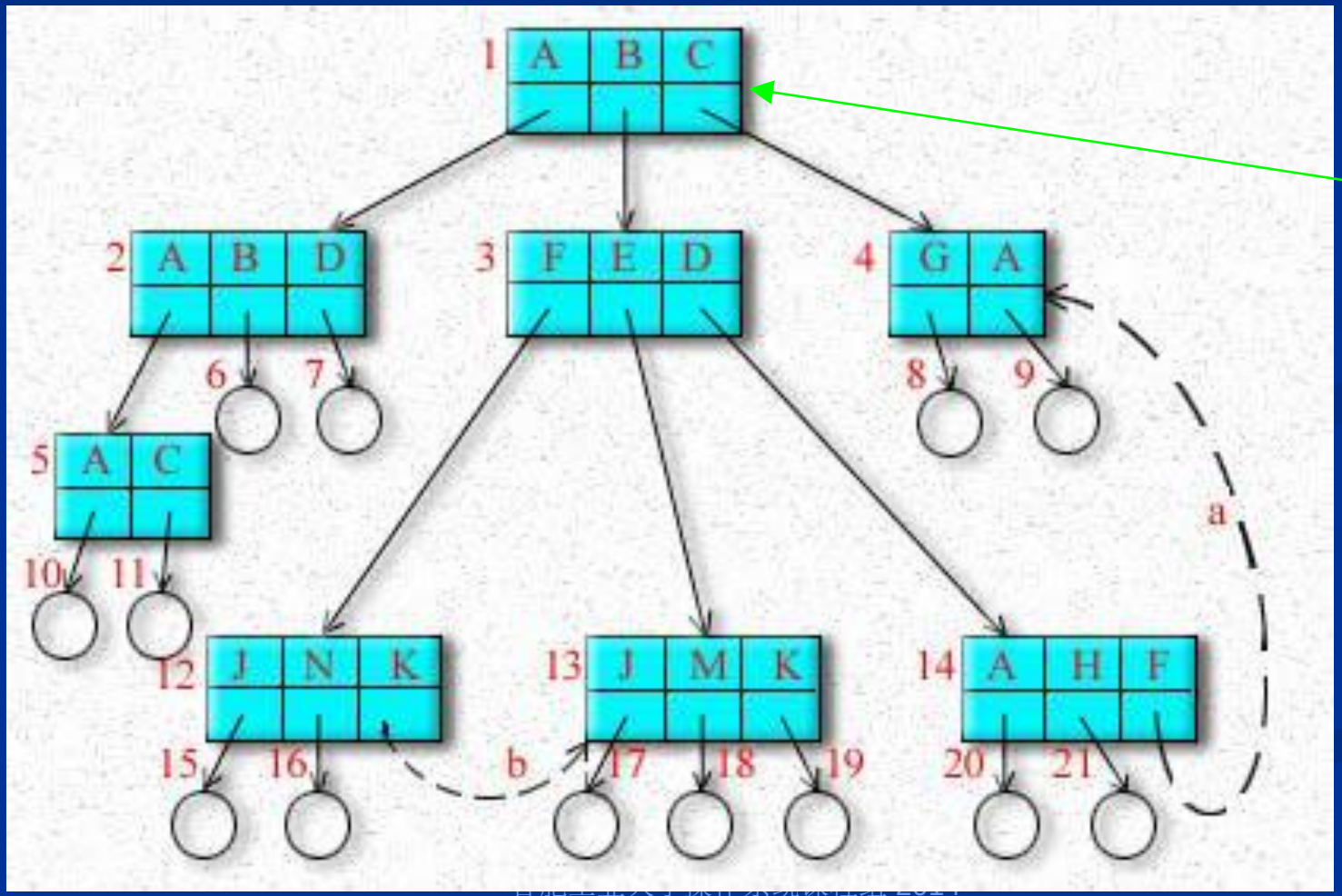
6.4.2 目录结构

- 特点
  - 改善目录检索速度；
  - 便于用户间文件重名；
  - 便于用户间文件共享；

6.4 目录管理

(3) 多级目录结构（树型目录结构）

允许目录下都可以建立子目录，文件可存放在任意子目录下。



6.4 目录管理

6.4.3 目录结构

- 特点
  - 层次结构清晰，便于管理和保护；
  - 有利于文件分类；
  - 解决重名问题；
  - 提高文件检索速度；
  - 能进行存取权限的控制
  - 查找一个文件按路径名逐层检查，由于每个文件都放在外存，多次访盘影响速度

6.4 目录管理

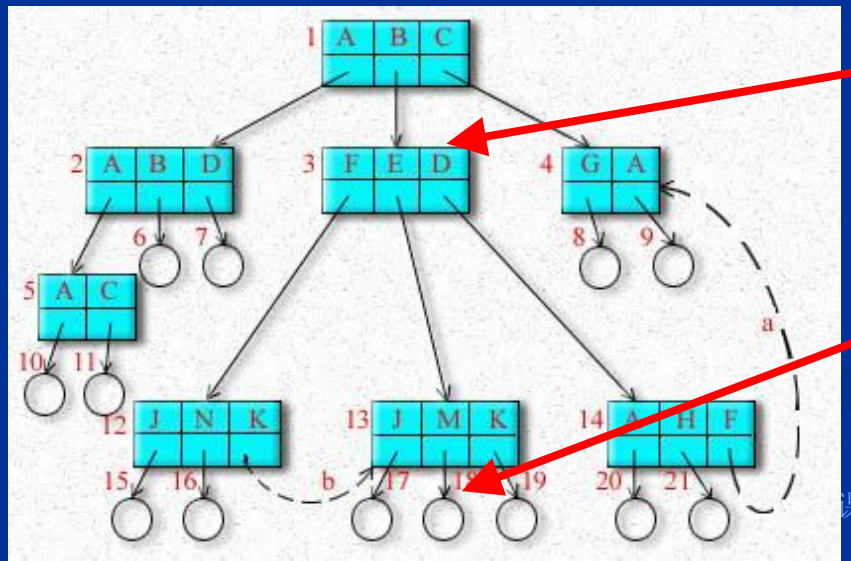
(3) 多级目录结构（树型目录结构）

- 路径名(path name)

以多级目录结构中某结点为起点，将到树中其他结点的路径上的目录名、文件名依次连接起来（用/或\隔开），即构成路径名。起点确定，路径名唯一。

此起点称为：当前目录。

- 绝对路径名：以根结点为起点的路径名。
- 相对路径名：以除根结点外的任意结点为起点的路径名。



当前目录：/B

文件名： M

相对路径名：E/M

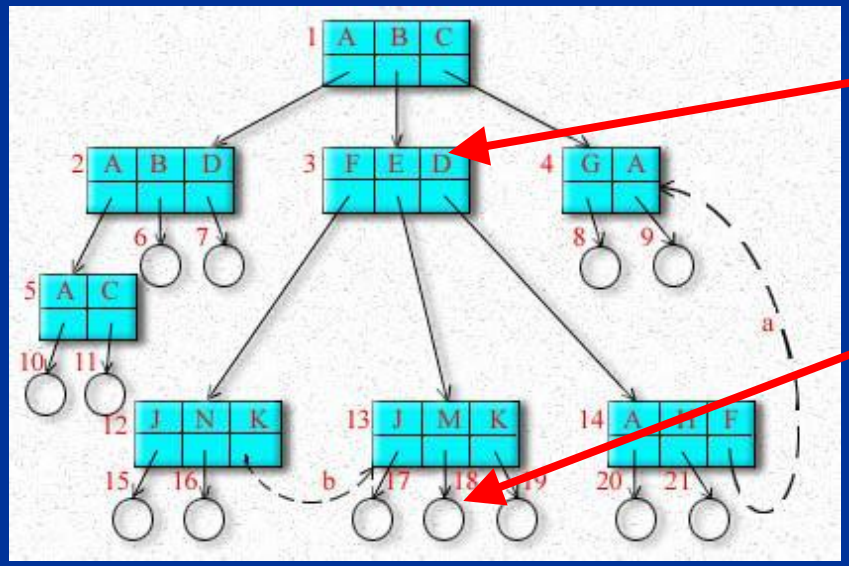
绝对路径名：/B/E/M

6.4 目录管理

6.4.3 目录查询技术（逻辑文件→物理文件映射）

- 线性检索法

分解路径名成各组成部分，然后依次查找各目录名、文件名。



当前目录：/B

文件名： M

相对路径名：E/M

绝对路径名：/B/E/M

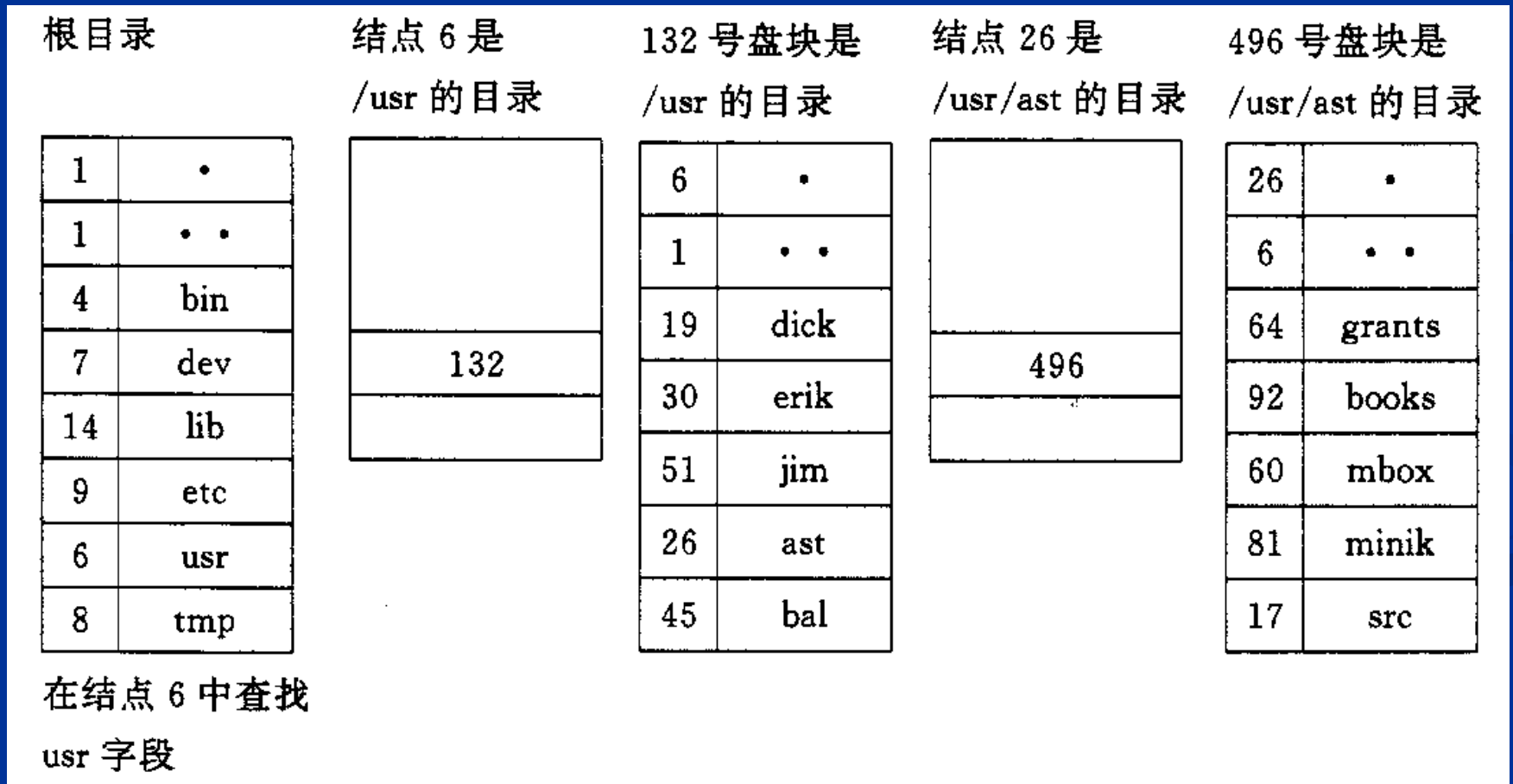
分解成4部分：/， B， E， M

6.4 目录管理

6.4.3 目录查询技术

- 线性检索法：基于索引结点的文件目录检索

检索：/usr/ast/mbox,过程如下：



6.5 文件存储空间管理

6.5.1 空闲表法

- 基本思想

  系统为外存所有空闲区建立一张空闲表，每个空闲区对应一个空闲表项。

序号	第一空闲盘块号	空闲盘块数
1	2	4
2	9	3
3	15	5
4	—	—

空闲盘块表

6.5 文件存储空间管理

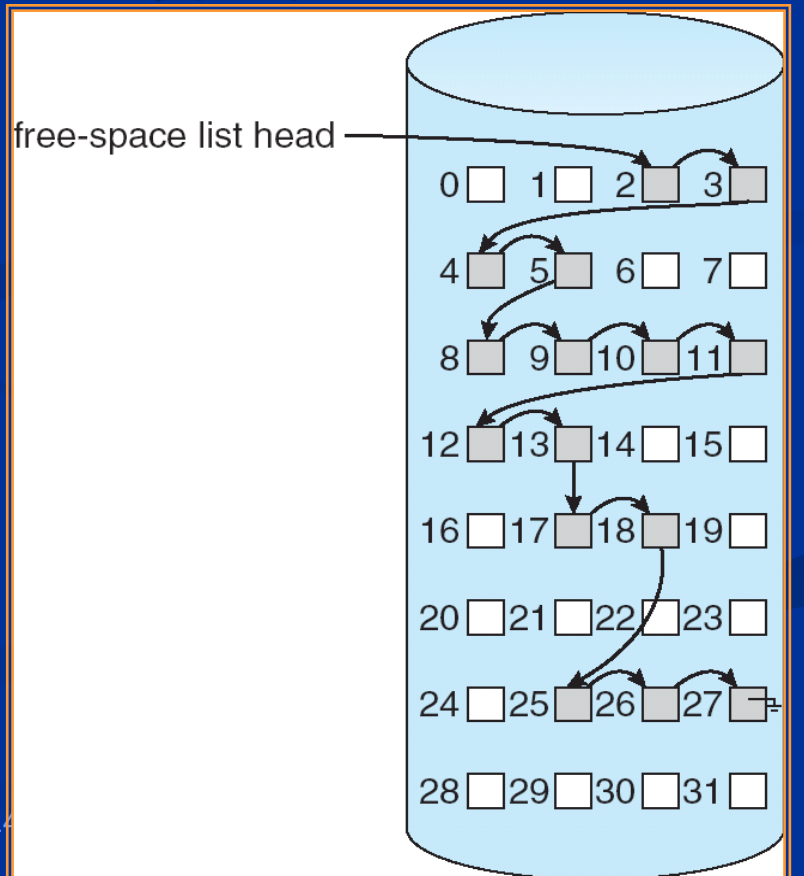
6.5.2 空闲链表法

- 基本思想

  系统为外存所有空闲区拉成一张空闲盘区（块）表。

  空闲盘块链：结点代表一个空闲盘块；

  空闲盘区链：结点代表一个空闲盘区；



6.5 文件存储空间管理

6.5.3 位示图法

- 基本思想

位示图利用二进制的一位来表示磁盘中一个盘块的使用情况。

- 当其值为“0”时，表示对应的盘块空闲；
- 为“1”时，表示已分配；

	0	1	2	3	4	…	15
0	1	1	1	1	0	1	1	0	0	0	0	1	0	0	0	1
1	1	1	1	1	0	1	1	0	0	0	0	1	0	0	0	1
2	0	0	0	1	0	0	0	0	1	1	1	1	1	1	1	1
3	1	0	0	1	1	0	1	0	1	0	1	1	0	0	0	0
4	1	1	1	1	0	1	1	0	0	0	0	1	0	0	0	1
5	1	1	1	1	0	1	1	0	0	0	0	1	0	0	0	1
6	0	0	0	1	0	0	0	0	1	1	1	1	1	1	1	1
7	1	0	0	1	1	0	1	0	1	0	1	1	0	0	0	0
	0	0	0	1	0	0	0	0	1	1	1	1	1	1	1	1

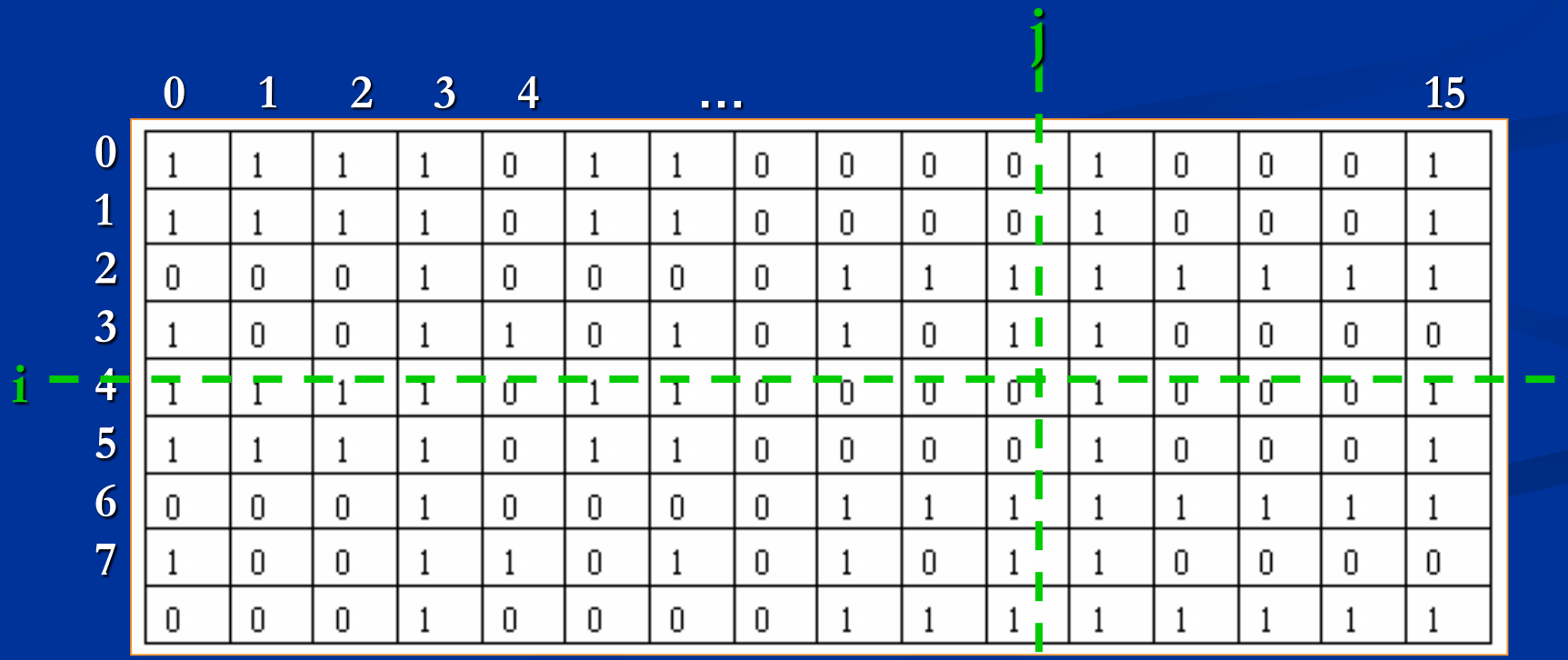
Var bitmap: array[1...m] of integer_16 ;

6.5 文件存储空间管理

- 磁盘块号与位示图行号、列号计算公式
  - 磁盘块号b → 位示图行号i、列号j（b, i, j均从0开始计数）:

    i = b/n ;　　j = b % n;　　// n为位示图宽度，常为字长
  - 位示图行号i、列号j → 磁盘块号b:

    b = i*n + j ;



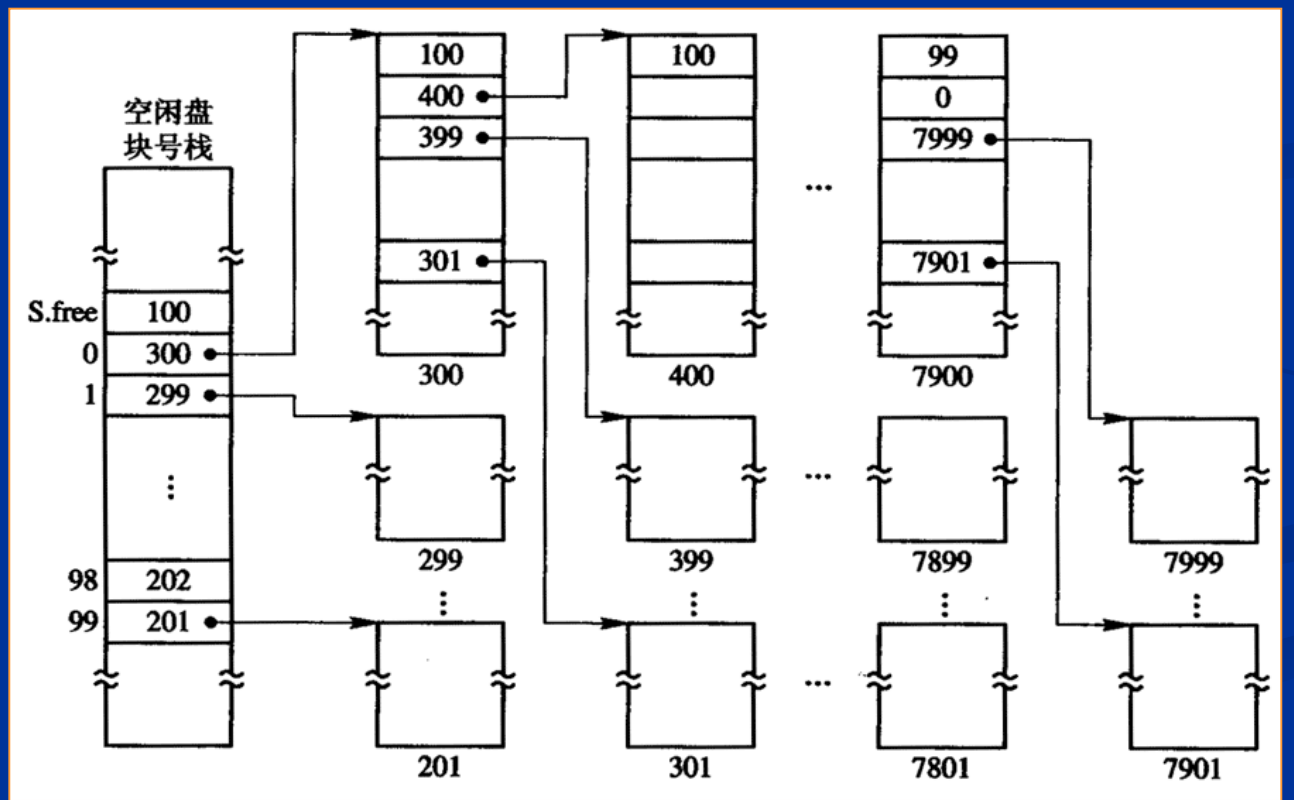
6.5 文件存储空间管理

6.5.4 成组链接法(UNIX)

- 基本思想

利用链栈存储所有的空闲磁盘块号。

链栈的每个结点存储一组（例如100个）磁盘块号，其中最后一个磁盘块号指向下一组所在的磁盘块。



6.5.4 成组链接法(UNIX)

- 相关算法
- 磁盘初始化

  (1) 内存设置空闲盘块号栈，可存储一组 (链栈一个结点所能存储的磁盘块号数量上限，例如100个)空闲磁盘块号；

  (2) 读磁盘上链栈的栈顶结点到内存，以初始化空闲盘块号栈。
- 盘块回收过程

  (1) 对于回收的磁盘块号，直接入内存空闲盘块号栈；

  (2) 如发现栈满，则将栈中数据(形成一组)写出到这个回收的磁盘块；

  (3) 清空内存空闲盘块号栈, 将回收的磁盘块号入栈。
- 盘块分配过程

  (1) 直接从内存空闲盘块号栈中，出栈一个磁盘块号，分配出去；

  (2) 如是栈中最后一个磁盘块号，则从外存调入下组磁盘块号（就在这个最后一个磁盘块内）；

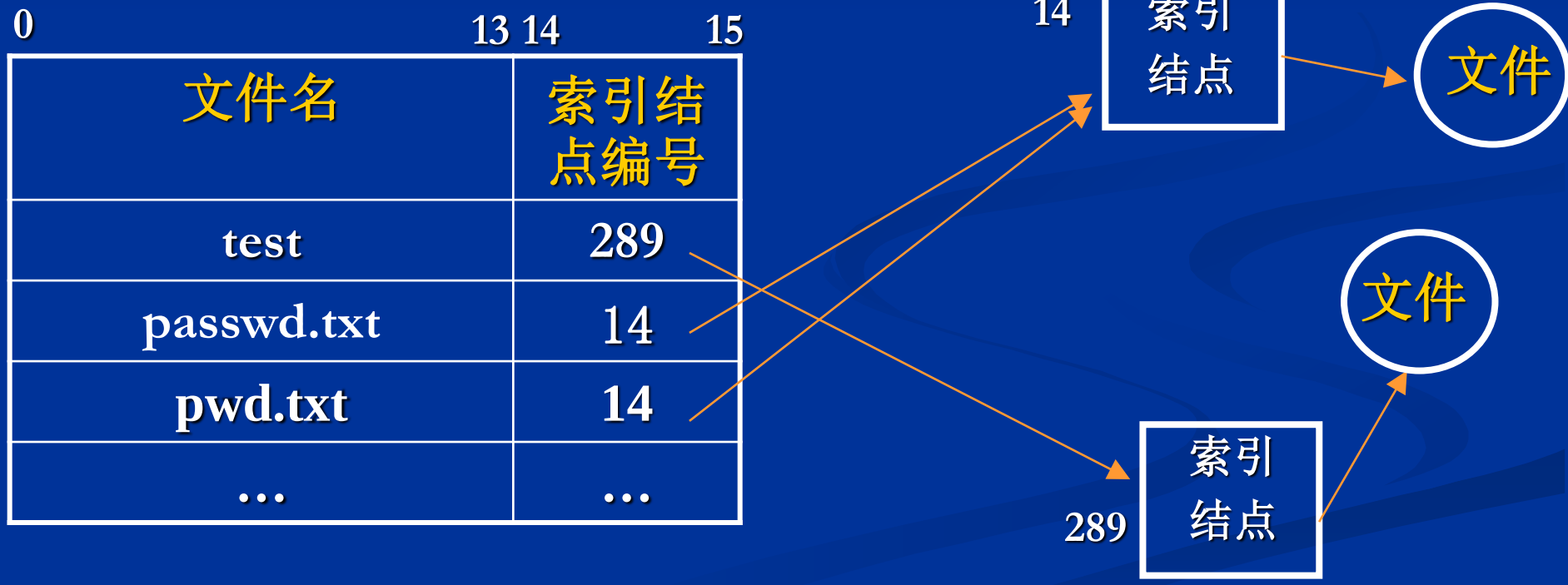
  (3) 分配这个最后一个磁盘块号。

6.6 文件共享

6.6.1 基于索引结点的共享

- 基本原理

  利用不同文件名指向相同索引结点。

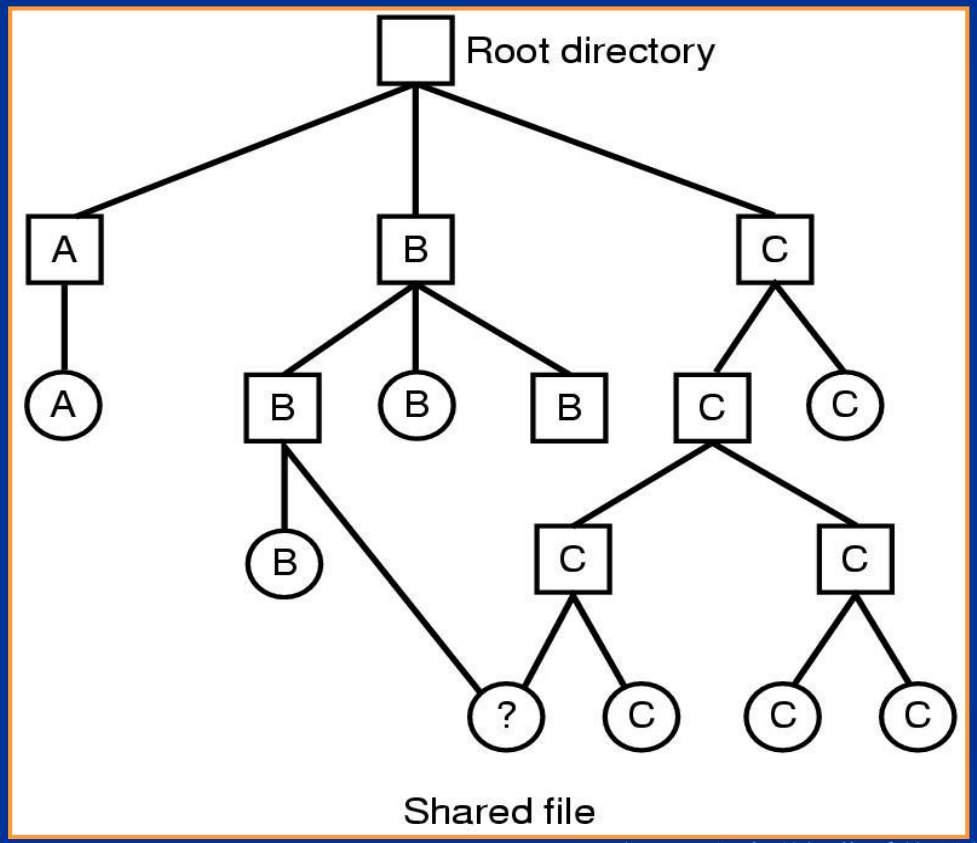


6.6 文件共享

6.6.1 基于索引结点的共享

- 基本原理

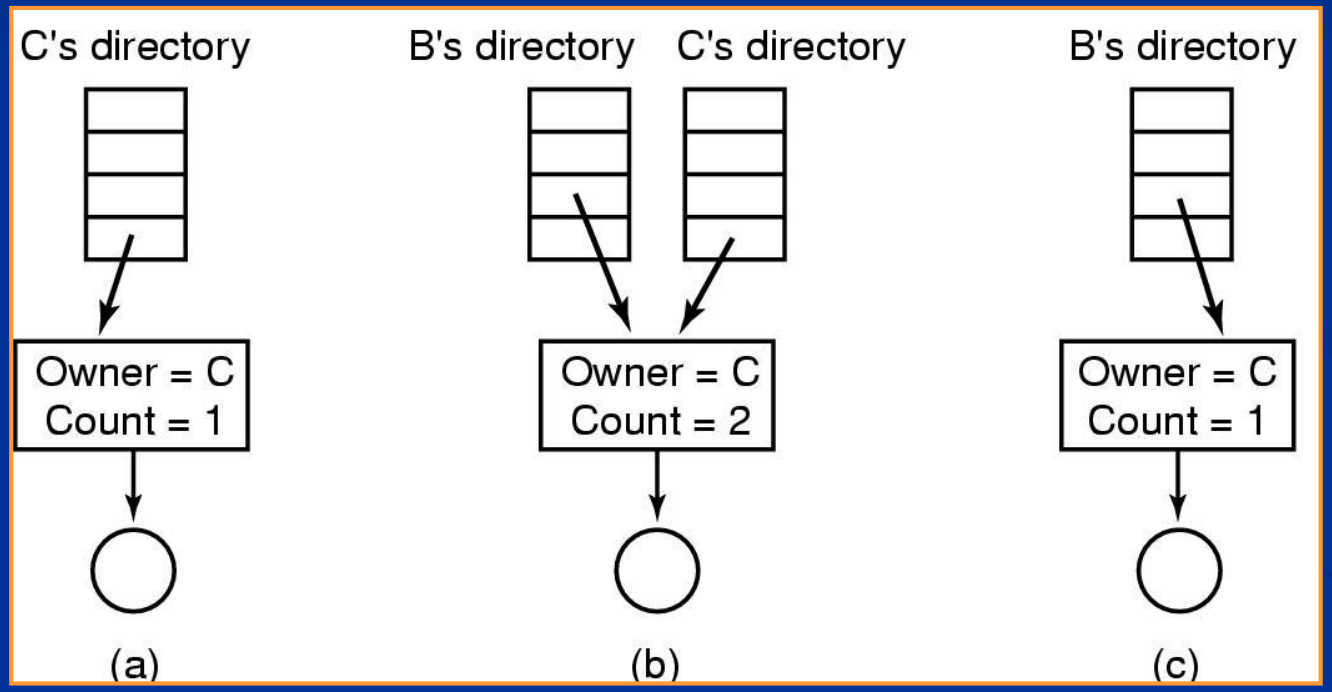
利用不同文件名或相同文件名（同一目录）指向相同索引结点。



6.6 文件共享

6.6.1 基于索引结点的共享

- 文件共享的过程



- 为何UNIX无直接删除文件的命令？

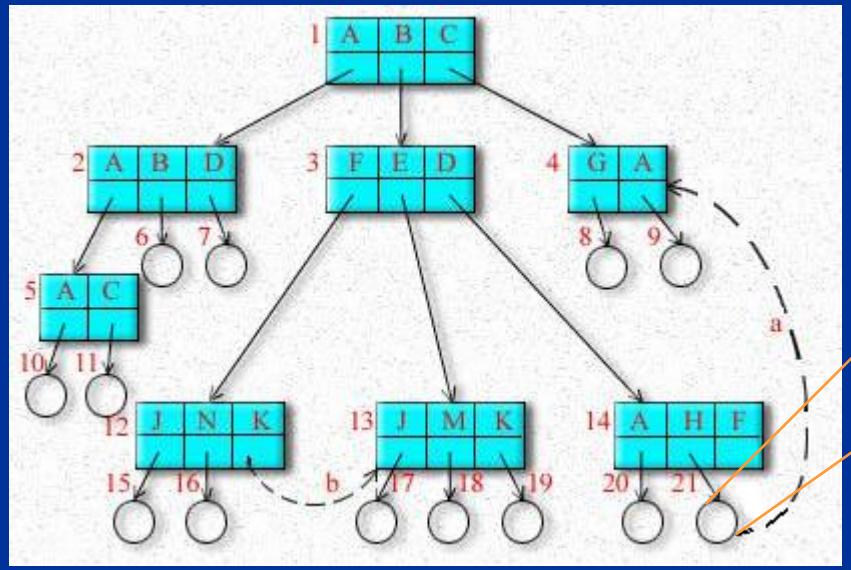
6.6 文件共享

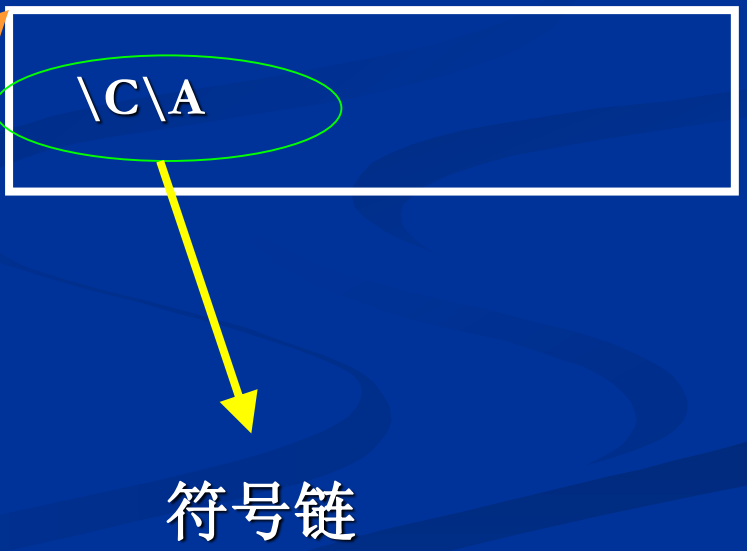
6.6.2 基于符号链的共享

- 基本原理

利用符号链文件，其中存储被共享的文件路径名称。

例如：Windows的快捷方式文件。





6.7 数据一致性控制

6.7.1 事务机制

- 程序的脆弱性

  一个示例：

```
cin >> No >> grade ;
ofstream  of1( “c:\\grades.txt” );
of1 << No ;
of1 << grade ;
ofstream of2( “c:\\details.txt”);
of2 << No ;
of1 << ( grade > 60 ? 1:0 );
```

6.7 数据一致性控制

6.7.1 事务机制

- 什么是事务

事务是一系列相关数据项的读写操作。

- 利用事务机制实现数据一致性控制的方法

① 设置事务日志(LOG)，按时间顺序存放事务记录，LOG存放在稳定存储器中；事务记录包括下列4类：

① 事务开始：　　开始事务，　<Ti, 开始>

② 读写操作：　　访问和修改记录，

③ 事务结束，正常：托付事务，　<Ti, 托付>

④ 事务结束，异常：夭折事务，　<Ti, 夭折>

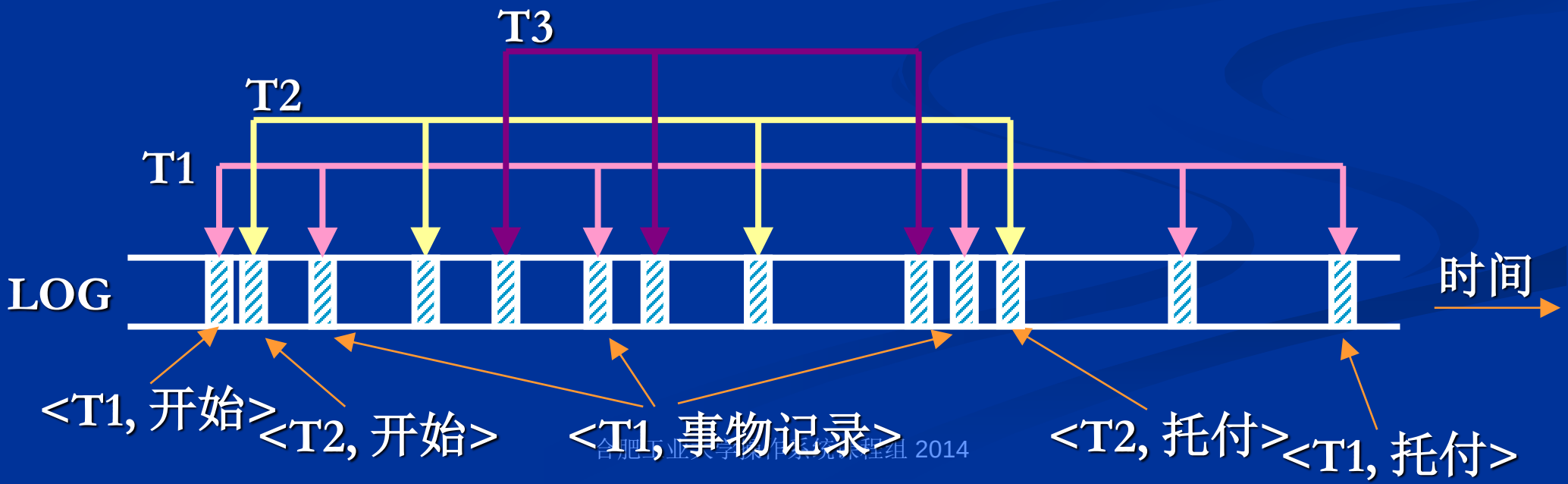
② 基本原则：

读写数据之前，必须先写事务日志

6.7.1 事务机制

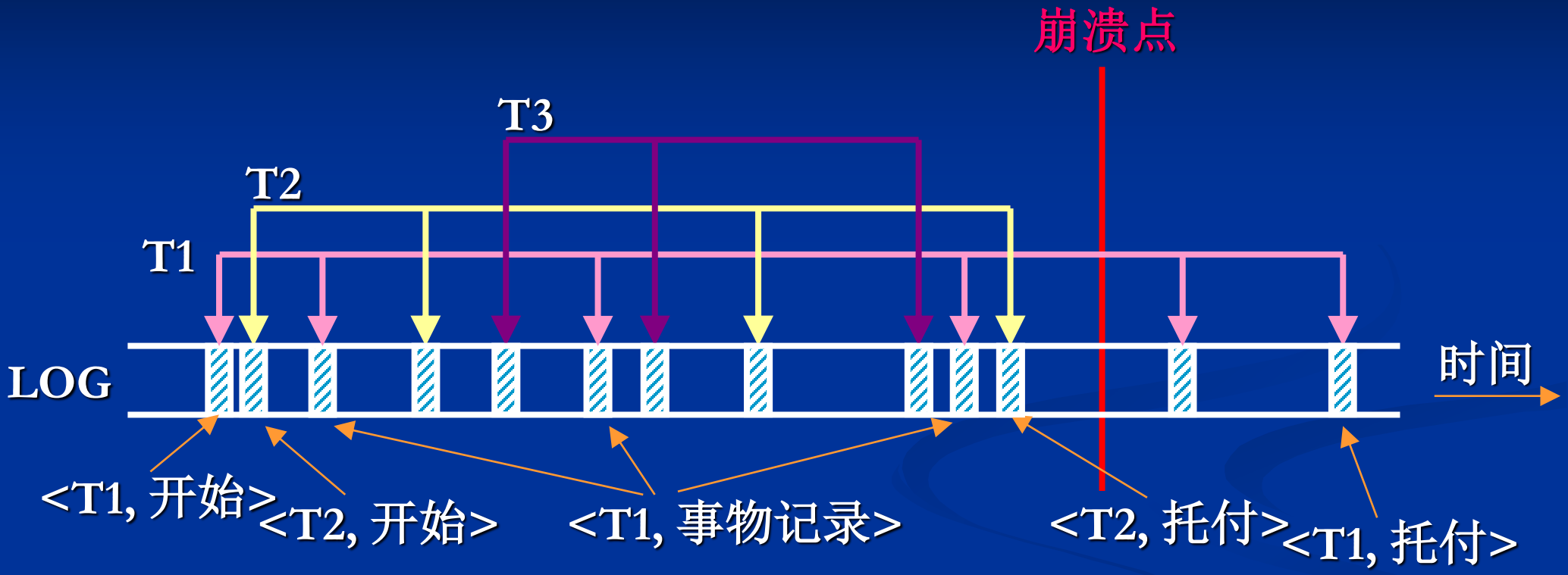
- 事务记录
  - ① 事务名：　事务的唯一标识；
  - ② 数据项名：被修改数据项的名称（唯一）；
  - ③ 旧值：　修改前数据项的值；
  - ④ 新值：　修改后数据项的值；

- 事务日志示例：



6.7.1 事务机制

- 事务的崩溃

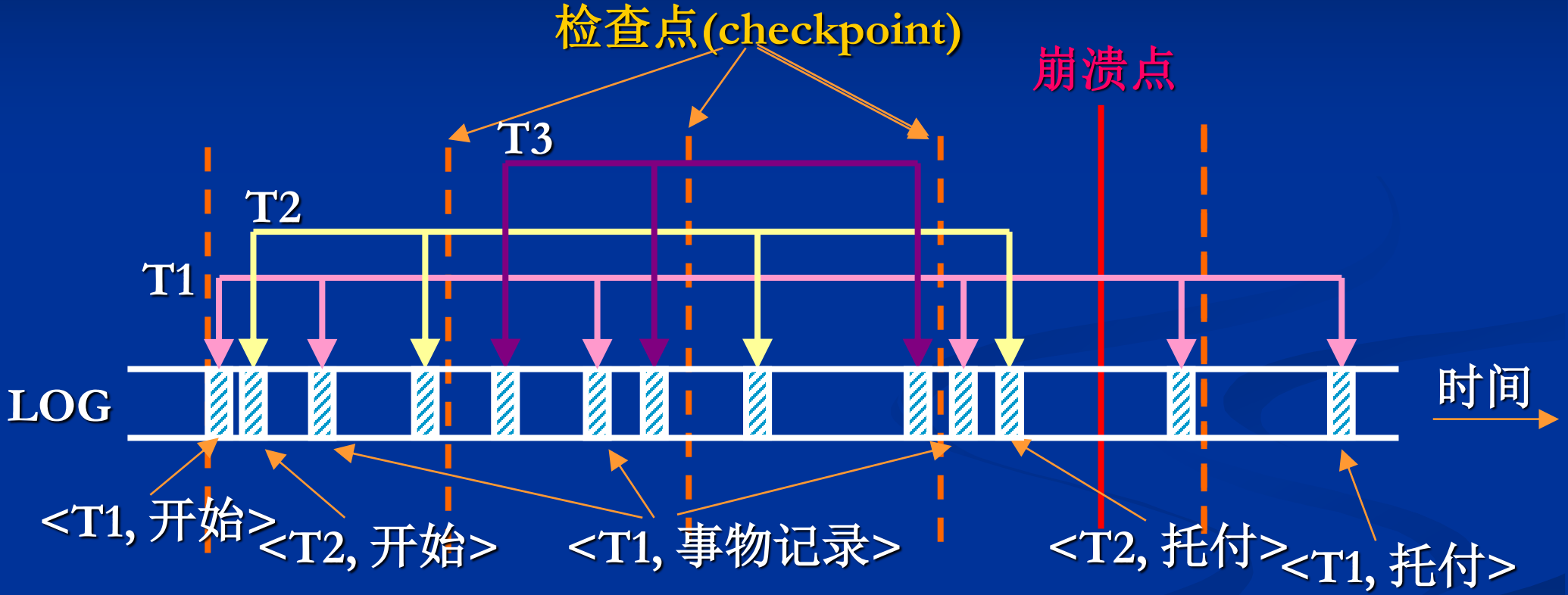


- 恢复算法
  - Undo<Ti>: 把所有被事务(无托付)修改后的数据项恢复为旧值;
  - Redo<Ti>: 把所有被事务(有托付)修改后的数据项重新置为新值;

6.7.1 事务机制

- 检查点机制

  设置检查点，定时刷新事务日志记录和被修改数据到稳定存储器。



- 恢复算法：可仅对最后一个检查点后的记录进行处理。
  - Undo<Ti>: 把所有被事务(无托付)修改后的数据项恢复为旧值；
  - Redo<Ti>: 把所有被事务(有托付)修改后的数据项重新置为新值；